
CSCI E-82 Advanced Machine Learning, Data Mining and Artificial Intelligence

Section 1: 5 September 2026, 11:00 AM EDT

Simili Abhilash

Contributor(s): *Daniel Sauble*

Welcome to CSCI E-82!

Topics for this section

- [Introduction and Python install](#)
 - [How to write a clean notebook](#)
 - [Principal Component Analysis](#)
 - [Appendix: Overview of Python, including Numpy, Pandas, and Matplotlib](#)
-

Sections

Sections for this course are optional, but recommended. In them, we will cover the practical aspects of the methods exposed in the lectures, and provide examples of code and directions where to turn for more guidance.

We hope that students will interact and engage with the discussion in Section - as we are using Zoom, it is easy to just speak up and participate. We welcome questions during the Section, especially if something is not clear.

Library

HOLLIS - https://hollis.harvard.edu/primo-explore/search?vid=HVD2&sortby=rank&lang=en_US

Python

Prerequisites and Expectations

- For this class, we will mostly be using **Python** for the Section material and for the assignments. The **Jupyter Notebook** (what you are looking at right now) is a browser-based application. Many of the homeworks submissions will require Jupyter Notebooks, though you may be able to incorporate .py files using any Python IDE of your choice. Jupyter Notebooks provide a simple way of presenting working code, its output, and your written answers/interpretation, all in one place.
 - Between the code, you can use Markdown to provide explanations; motivate what you are doing; describe what results you are getting; and provide your conclusions.
 - If you have not taken any programming courses at Harvard Extension School (or similar) then this course will likely be extremely challenging. You should be able to download and parse data files such as CSV or JSON formats, use control loops and recursion. Experience with Python in particular is not essential but will be helpful.
 - You should already have experience with other languages (java, C"*, etc.), or, if you've done any type of scientific programming with another language (Matlab, R, etc.) and you're comfortable learning a new language on your own, then that should suffice.
 - If you've completed [CSCI E-109 A/B](#) then you should be well prepared, as far as Python is concerned. But if Python is new to you then [here is a link](#) to some great Python tutorials to help get you started.
 - In addition to Python, you should be comfortable with:
 - reading API's for libraries like [scikit-learn](#) or other [documentation](#)
 - searching websites like [Stackoverflow](#) for help to common questions
 - posting and/or answering questions on Ed Discussion, which can be found by going into Canvas.
-

Installing Python

- Python is free and comes distributed with most Unix/Linux based systems pre-installed (distributions of Linux, but also Mac OS X). **Please note: we will be using Python >= 3.13 in this section - if you have to use an older version, please be ready to adjust for incompatibilities. We**

recommend to upgrade if you can, so that we are all on the same page.

- There are also many free third-party libraries available for scientific computing. We could list all the useful libraries here and install them individually, but **Continuum/Anaconda** has done a nice job of that for us already (mostly) with their Anaconda distribution. If you don't already have it, the [download and install it from here](#).
- When you have finished installing the distribution, open a terminal and execute the following commands to get the latest versions of all the libraries. Note that you **must** have a working internet connection for this to be successful.
 1. `conda update conda`
this updates the conda package manager
 2. `conda update --all`
this updates all installed packages to the latest, but respecting inter-compatibilities

Conda Environments

See the [documentation](#) for this, since conda can save you a great deal of time.

You can also use the conda package manager to create or clone Python environments. Below are some example commands from the Anaconda documentation.

```
conda create -n py313env python=3.13
```

or with specific versions and packages (only) in the environment

```
conda create -n slim_py35env python=3.5 scipy=0.16.0 matplotlib numpy
```

and it can also be used to upgrade your installation of Python

```
conda create -n py310 python=3.10 anaconda
```

or with just Python

```
conda install python=3.13
```

But - a major gotcha - is to remember to activate the right environment before using it, otherwise Python will start telling you it does not have what you think it does.

```
conda activate py313env
```

Tip: efficient, reproducible environment management

You may have heard the saying that data science is 80 percent data cleaning. While that can be true at times, you may also find that environment setup and management can be a significant chunk of the time for any project if you face challenges setting up your environment. Since each person's local environment can be different due to the time a notebook was first built, as well as differences between systems themselves, it's useful to learn the skill of how to do this.

With the above commands, you can stand up bespoke virtual environments for use in a single notebook very easily. By copying code from one notebook to another, you can reproduce the environment you had for a given project. However, as you gain more experience with Python and Linux, you will likely find yourself reusing most of the same code over and over again. Moreover, one of the **key benefits of installing from conda is that it works through dependencies between packages for you (most of the time)**. You will see this best if you set up your environment from the command line. Notice how it spends a lot of time solving for the environment before completing the installation. If you do enough environment installations, you'll also begin to notice that sometimes you don't seem to always get to a successful installation using the same command. There are at least two reasons for this: (1) order of the installation - if installing packages in series, one after another, you may be setting up dependencies as you go that are not satisfiable; (2) lack of specificity with a constraining dependency - an environment might only be solvable if you specify a specific version.

The best method to remedy these types of issues is to build and test your environments more systematically than doing this one command at a time. We demonstrate a framework for this that will enable you to very rapid environment setup and debugging. This technique is especially useful for managing GPU aware installations on Linux, for example. Typical steps:

- (1) **Identify the packages** you think you'll need - you probably won't know this the first time through a notebook. It's very common to realize that you want some extensibility during a project and to add it then;
- (2) **Construct an environment file** that contains all of the dependencies you need through your notebook - over time, I have worked out a system for this and I reuse, create new versions, and repurpose environments across different projects so that I manage that knowledge effectively;
- (3) **Decide on success criterion for your environment** - always, you want the packages to run. For GPU aware versions of your project, an additional success criterion is that you would want your libraries to be able to process commands through a GPU so that the notebook runs much faster - essential for serious, iterative deep learning;
- (4) **Run the environment setup** - you will likely find that, for certain packages to work, you will need to fine tune this file a bit to satisfy all the dependencies you need. Handling this up front, and circling back as you run through your notebook when you find a new package requirement will save you time and frustration later.
- (5) **Test the environment** - typically done by running the notebook. For example, run commands from the library you're working with to verify that the new environment is GPU aware. Doing so up front before doing anything else has a good return on time since CPU driven deep learning takes quite a bit longer to run.

Reference: <https://docs.conda.io/projects/conda/en/latest/user-guide/tasks/manage-environments.html>

Opening a terminal to do the installation, vs trying to do everything from bash commands in the notebook, has the advantage of giving more interactive capabilities during the installation process. If you really want to, you can refine the command to answer all of the yes or no questions you are asked how you want them, but this is not necessary to execute the process well.

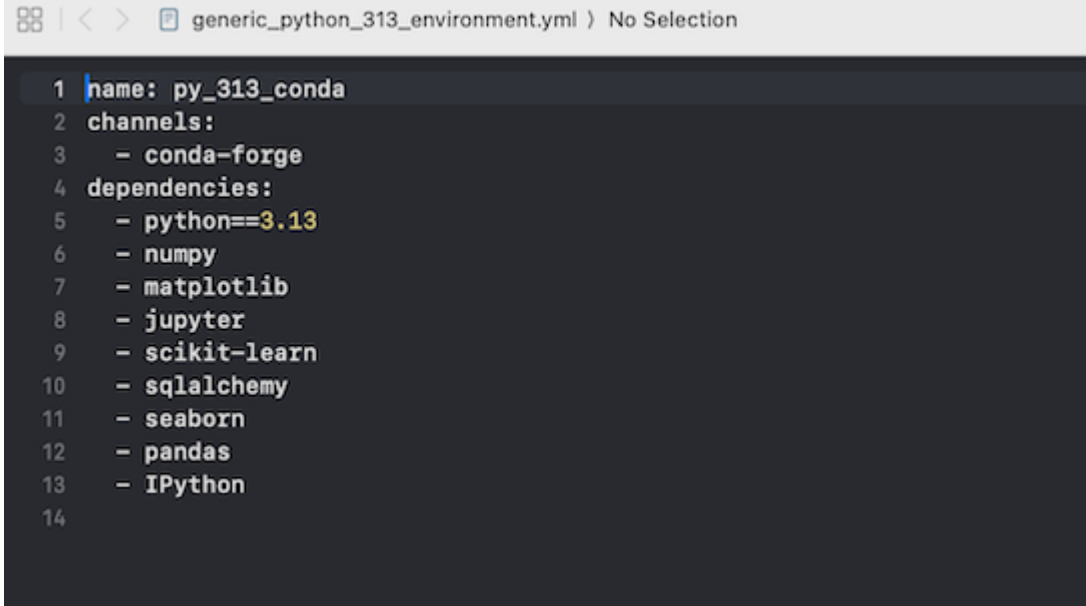
One environment to rule them all?

A common starting place is to try to just do environment setup as a one time type of activity. Most of the time, there will be some differences between the package requirements to support various efforts. When that is the case, the way to control for those changes is to repeat this process with a new environment. You may want to do this for each new assignment.

Below is an example of a very simple environment setup. The yaml file format is very simple but Visual Studio Code has a very useful plugin to help with code checking.

```
In [1]: from IPython import display  
  
display.Image("example_yaml.png")
```

```
Out[1]:
```



Navigate to the directory where your yaml setup file is and run the anaconda command to run the environment setup from the configs in the yaml file:

```
conda env create -f generic_python_313_environment.yml
```

Note that the name of the file does not have to be the same as the name of the environment. Notice how it works through all of the particulars for the whole environment and installs it.

```
In [2]: display.Image("example_install.png")
```

```
Out[2]: [(base) ~/Documents/TA/Advanced-ML-2026/Section1 >conda env create -f generic_python_313_environment.yml
2 channel Terms of Service accepted
Channels:
- conda-forge
- defaults
Platform: osx-arm64
Collecting package metadata (repodata.json): done
Solving environment: done

## Package Plan ##

environment location: /opt/anaconda3/envs/py_313_conda

added / updated specs:
- ipython
- jupyter
- matplotlib
- numpy
- pandas
- python==3.13
- scikit-learn
- seaborn
- sqlalchemy

The following NEW packages will be INSTALLED:
```

Note: If you run into any messages during this process, you should pay attention to them - things like: older version of anaconda, package conflicts, etc. All of these things will happen no matter how you do your environment setup, but this way you are able to see what's going on and judge and manage risks for yourself systematically up front. The above is what your terminal will look like when anaconda is up to date and there are no package conflicts. Most of the time, it'll be easy to get here but this process will help you for when you run into friction with an environment setup.

When you don't get this:

Remove the environment - conda will not let you install over an environment and even a partial install will create a place on disk for the name of the environment. **Note** that the environment has to be removed by its name - not the name of the file used to install it:

```
conda env remove --name py_313_conda
```

Note of caution:

When installing python on your machine, you should **always** use a virtual environment. The reason for this is that, particularly with Linux and Mac, your base operating system (Linux, MacOS) uses a version of Python. When you don't use a virtual environment, you are making changes to your base operating system. While this might sound benign, you are creating risks to your operating system that might cause you to need to reinstall your operating system - which could be very inconvenient in the middle of an assignment. A virtual environment creates logical separation between the python in your base OS and your project - which is the most stable strategy for managing all of the complexity with this process.

Pip for installing packages not in Anaconda

There are packages that we will use that are not part of the Anaconda distribution. For these, the easiest thing is to use `pip install` to install them into whatever python environment you want (remember to activate it first). e.g:

```
python -m venv test
cd test
source ./bin/activate
python -m pip install tsfresh
deactivate
```

You can actually use just `pip` (instead of `conda`) as a package manager. This is sometimes useful when all you want is a barebones Python machine with specific packages - for example, when spinning up an AWS EC2 instance to complete a machine learning task.

Jupyter Notebook

We will primarily use the Jupyter Notebook for code assignments. Jupyter Notebook is a browser-based "interactive computational environment" that allows you to "combine code execution, rich text, mathematics, and plots". To start the notebook server, open a terminal and type

```
jupyter notebook
```

To get started on this homework, download a local copy of this notebook and save it to a directory. Then browse to that directory in your terminal and execute the command above to begin working.

The notebook consists of a sequence of cells. A cell is a multiline text input field, and its contents can be executed by using **Shift-Enter**, or by clicking either the "Play" button the toolbar, or Cell | Run in the menu bar.

Keyboard Shortcuts - On the Menu bar **Help -> Keyboard Shortcuts** [Some common helpful shortcuts (OSx) ESC - A to add cell above, ESC - B add cell below, Shift-Enter - Run a cell

Converting this notebook to PDF

Install the following packages:

- jupyter and nbconvert (`pip install jupyter nbconvert`)
- Install pandoc - <http://pandoc.org/installing.html>
- Install MacTeX - <http://tug.org/mactex/> (~4GB file on Mac)

Now, to export your notebook to PDF, run this command in your terminal:

```
jupyter nbconvert --to PDF your-notebook-name.ipynb
```

(Make sure all your code is there in PDF and not getting cut off after ~n characters)

Note: If pdf export fails, you can also go to File > Save and Export Notebook As HTML, then open the HTML in your browser and save as PDF.

When conda environments doesn't show up on Jupyter Kernel list

When conda environments doesn't show up on Jupyter Kernel list - <https://stackoverflow.com/questions/39604271/conda-environments-not-showing-up-in-jupyter-notebook>

[back to top](#)

How to write a clean notebook

Notebooks are more than a place to write and run code. They're a way to communicate your ideas and iterate on experiments so that someone else can understand what you're doing and follow along.

- Show your work and the results (be generous with comments)
- Write clear explanations of what you tried and why (if you tried something that didn't work, tell us why you think it didn't work and what you're going to try next)
- Avoid extra long lines of code (these often get cropped out when you export to PDF)
- Try to write code in a linear way (imagine someone reading your notebook from top to bottom, can they follow along?)
- Tools are sometimes too verbose in the output they produce, which clogs up a notebook and makes it hard to follow along. Try to filter this out.
- Make sure all datasets and other assets are included in your submission. Graders should be able to open a notebook, hit run, and have everything just work.

Here's an example of a **code block** which isn't very clear:

```
In [3]: def find(start, end): # Method lacks a clear name and description, and doesn't show the iterations you took to get here
primes = []
for x in range(start, end):
    print(f'Is {x} prime?') # Excess debug statements
    if x < 2:
        print(f'{x} is not prime')
        continue
    is_prime = True
    for y in range(2, x - 1):
        if x % y == 0:
            print(f'{x} % {y} == {x % y}') # Unclear what code is doing
            is_prime = False
            break
    if is_prime:
        print(f'{x} is prime')
        primes.append(x)
    else:
        print(f'{x} is not prime')

return primes

primes = find(0, 20) # Results not printed
```

Is 0 prime?
0 is not prime
Is 1 prime?
1 is not prime
Is 2 prime?
2 is prime
Is 3 prime?
3 is prime
Is 4 prime?
4 % 2 == 0
4 is not prime
Is 5 prime?
5 is prime
Is 6 prime?
6 % 2 == 0
6 is not prime
Is 7 prime?
7 is prime
Is 8 prime?
8 % 2 == 0
8 is not prime
Is 9 prime?
9 % 3 == 0
9 is not prime
Is 10 prime?
10 % 2 == 0
10 is not prime
Is 11 prime?
11 is prime
Is 12 prime?
12 % 2 == 0
12 is not prime
Is 13 prime?
13 is prime
Is 14 prime?
14 % 2 == 0
14 is not prime
Is 15 prime?
15 % 3 == 0
15 is not prime
Is 16 prime?
16 % 2 == 0
16 is not prime
Is 17 prime?
17 is prime
Is 18 prime?

```
18 % 2 == 0
18 is not prime
Is 19 prime?
19 is prime
```

And here's a **better version** of the same code:

```
In [4]: import time

# Attempt 1: Generate an array of prime numbers by testing all
# numbers between 1 and each candidate number (exclusively)
def find_primes_exhaustively(start, end):
    primes = []
    for x in range(start, end):

        # The only even prime number is 2, so we'll make this a
        # special case
        if x < 2:
            continue

        # Assume each number is prime initially
        is_prime = True

        # Prime numbers are only divisible by 1 and themselves, so
        # check all numbers within that range
        for y in range(2, x - 1):

            # If the number is evenly divisible, it's not prime
            if x % y == 0:
                is_prime = False
                break

        # Add the number to the list of primes if it's prime
        if is_prime:
            primes.append(x)

    return primes

# Generate an array of primes between 0 and 1000
start_time = time.time()
primes = find_primes_exhaustively(0, 1000)
end_time = time.time()

# Show the results
print(primes)
print(f"Found {len(primes)} primes in {end_time - start_time} seconds")
```

```
# This code works but generates primes fairly inefficiently. Let's try an alternative algorithm which  
# takes advantage of the fact that we only need to evaluate candidate numbers against the existing  
# set of prime numbers that are less than the candidate we are testing.
```

```
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97, 101, 103, 107, 109, 113, 127, 131, 137, 139, 149, 151, 157, 163, 167, 173, 179, 181, 191, 193, 197, 199, 211, 223, 227, 229, 233, 239, 241, 251, 257, 263, 269, 271, 277, 281, 283, 293, 307, 311, 313, 317, 331, 337, 347, 349, 353, 359, 367, 373, 379, 383, 389, 397, 401, 409, 419, 421, 431, 433, 439, 443, 449, 457, 461, 463, 467, 479, 487, 491, 499, 503, 509, 521, 523, 541, 547, 557, 563, 569, 571, 577, 587, 593, 599, 601, 607, 613, 617, 619, 631, 641, 643, 647, 653, 659, 661, 673, 677, 683, 691, 701, 709, 719, 727, 733, 739, 743, 751, 757, 761, 769, 773, 787, 797, 809, 811, 821, 823, 827, 829, 839, 853, 857, 859, 863, 877, 881, 883, 887, 907, 911, 919, 929, 937, 941, 947, 953, 967, 971, 977, 983, 991, 997]  
Found 168 primes in 0.0027141571044921875 seconds
```

In [5]: *# Attempt 2: Generate an array of prime numbers more efficiently by
only checking against previously found prime numbers*

```
def find_primes_efficiently(end):  
    primes = []  
    for x in range(end):  
  
        # There are no primes smaller than 2  
        if x < 2:  
            continue  
  
        # The only even prime number is 2, so we'll use this as our  
# seed prime for finding other primes  
        if x == 2:  
            primes.append(x)  
            continue  
  
        # Assume each number is prime initially  
        is_prime = True  
  
        # Is a number is not divisible by any prime number smaller  
# than it is, it is prime  
        for y in primes:  
  
            # If the number is evenly divisible, it's not prime  
            if x % y == 0:  
                is_prime = False  
                break  
  
        # Add the number to the list of primes if it's prime  
        if is_prime:  
            primes.append(x)  
  
return primes
```

```
# Generate an array of primes up to 1000
```

```
start_time = time.time()
```

```
primes = find_primes_efficiently(1000)
```

```
end_time = time.time()
```

```
# Show the results
```

```
print(primes)
```

```
print(f"Found {len(primes)} primes in {end_time - start_time} seconds")
```

```
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97, 101, 103, 107, 109, 113, 127, 131, 137, 139, 149, 151, 157, 163, 167, 173, 179, 181, 191, 193, 197, 199, 211, 223, 227, 229, 233, 239, 241, 251, 257, 263, 269, 271, 277, 281, 283, 293, 307, 311, 313, 317, 331, 337, 347, 349, 353, 359, 367, 373, 379, 383, 389, 397, 401, 409, 419, 421, 431, 433, 439, 443, 449, 457, 461, 463, 467, 479, 487, 491, 499, 503, 509, 521, 523, 541, 547, 557, 563, 569, 571, 577, 587, 593, 599, 601, 607, 613, 617, 619, 631, 641, 643, 647, 653, 659, 661, 673, 677, 683, 691, 701, 709, 719, 727, 733, 739, 743, 751, 757, 761, 769, 773, 787, 797, 809, 811, 821, 823, 827, 829, 839, 853, 857, 859, 863, 877, 881, 883, 887, 907, 911, 919, 929, 937, 941, 947, 953, 967, 971, 977, 983, 991, 997]
```

```
Found 168 primes in 0.0005600452423095703 seconds
```

This example is perhaps a bit heavy on the comments. You don't have to add comments to this extent, but it's better to err on the side of more comments since it demonstrates that you know what you're doing and requires less interpretation on the part of the person reading your notebook.

[back to top](#)

Matrix Operations

<https://numpy.org/doc/stable/reference/generated/numpy.ndarray.html>

Addition

Both matrices have to have the same size

Output has the same size as inputs

The sum at any position is the sum of the same position of the matrix values at that position

```
In [6]: import numpy as np;
A = np.array([[3, 8],
              [4, 6]])

B = np.array([[4, 0],
              [1, -9]])
```

```
print ("A+B = \n", A+B)
type(A)
```

```
A+B =
[[ 7  8]
 [ 5 -3]]
```

Out[6]: numpy.ndarray

Multiplication

AxB matrix times BxC matrix -> AxC matrix

e.g., 3x2 matrix (X in example below) times 2x1 matrix (Y) -> 3x1 matrix

Number of columns in X (first matrix) must equal the number of rows in Y (second matrix)

Multiply each row elements of X with the corresponding index element in Y, and add

```
In [7]: X = np.array([[3, 8],
                     [4, 7],
                     [5, 6]])

Y = np.array([[10],
              [100]])

print ("X @ Y = \n", X @ Y) # use @ for matrix multiplication
```

```
X @ Y =
[[830]
 [740]
 [650]]
```

Transpose

Matrix A is mxn A' or At or AT will be nxm

Every position (i,j) -> (j,i)

```
In [8]: A = np.array([[3, 8],
                     [4, 7],
                     [5, 6]])

print ("A' = \n", A.T)
```

```
A' =
[[3 4 5]
 [8 7 6]]
```

Principal Component Analysis (PCA)

Why PCA:

- Reduce dimensionality of the dataset with minimal loss of information.
- Compress data without much loss of information, e.g., image compression
- As a first step in Feature engineering, Principal Component Regression, to quick rank observations,...

Principal Components:

- A subset of ordered abstract projection that can best **preserve the variations and correlations** of the original data
- A dataset with n instances and p features (p -dimensional data) can be approximately reproduced using $k < p$ Principal Components

Mathematically:

- An eigendecomposition of the covariance matrix
- Finds directions that maximize the variance in the dataset.
- In PCA, we are projecting the entire set of data onto a different subspace
- In PCA, we are trying to find the axes with maximum variances where the data is most spread

PCA can be done by eigenvalue decomposition of a data covariance (or correlation) matrix or singular value decomposition of a data matrix, usually after a normalization step of the initial data. [Difference between eigen decomposition and SVD decomposition](#)

We might be interested in keeping only those eigenvectors with the largest eigenvalues, since they contain more information about the data.

Create new feature subspace:

- eigenvalues that are close to 0 are less informative: they are dropped when we construct new feature subspace.

Algorithms for PCA

- Load a dataset that consists of p features
- Compute the covariance matrix of the data
- Compute the eigenvectors and the corresponding eigenvalues
- Sort the eigenvectors by decreasing eigenvalues
- Select the top k eigenvectors with the largest eigenvalues to form $p \times k$ matrix
- Use the $p \times k$ eigenvector matrix to transform the sample to a new subspace.

Step-by-step Examples

- PCA (with no sklearn)
- US Arrests data (with and without sklearn)

PCA function in Python ecosystem

- [Scikit-learn: sklearn.decomposition module](#)
- For visualizing - [matplotlib](#) or [seaborn](#)

Other interesting/good PCA references -

- [Visualization](#)
- [ISLR Book - Chapter 10](#)
- [PCA-Wikipedia](#)
- PCA Playlist - https://www.youtube.com/playlist?list=PLBv09BD7ez_5_yapAg86Od6JeeypkS4YM

Intuition - a step-by-step example using 2D data points based on a tutorial on pca

<https://www.cs.otago.ac.nz/research/publications/OUCS-2002-12.pdf>

- Some code is given in Scilab (free alternative to Matlab)
- We will use Python and associated libraries

Step 1: Create and plot the raw data

In [243...

```
import numpy as np
x = [-0.4, -0.3, 0.3, 2.5, 0.5, 2.2, 1.9, 3.1, 2.3, 2, 1, 1.5, 1.1]
y = [-0.7, -0.4, 0., 2.4, 0.7, 2.9, 2.2, 3.0, 2.7, 1.6, 1.1, 1.6, 0.9]
data = np.array([x,y]).T
```

```
data
```

```
Out[243...] array([[ -0.4, -0.7],
                [ -0.3, -0.4],
                [  0.3,  0. ],
                [  2.5,  2.4],
                [  0.5,  0.7],
                [  2.2,  2.9],
                [  1.9,  2.2],
                [  3.1,  3. ],
                [  2.3,  2.7],
                [  2. ,  1.6],
                [  1. ,  1.1],
                [  1.5,  1.6],
                [  1.1,  0.9]])
```

```
In [244...] import pandas as pd
df = pd.DataFrame(data, columns=['x', 'y'])
print(df.shape)
df.head()
```

```
(13, 2)
```

```
Out[244...]      x    y
0 -0.4 -0.7
1 -0.3 -0.4
2  0.3  0.0
3  2.5  2.4
4  0.5  0.7
```

```
In [245...] # This is just a helper function to get a plot which is centered on the origin in Matplotlib

set2_colors = ['#66c2a5', '#fc8d62', '#8da0cb', '#e78ac3', '#a6d854', '#ffd92f', '#e5c494', '#b3b3b3']
light_grey = np.array([float(248)/float(255)]*3)
shade_black = '#262626'

#What are spines, ticks, labels, etc - https://matplotlib.org/3.1.1/gallery/showcase/anatomy.html

def cible_border(axes=None, top=False, right=False, left=True, bottom=True):
    """
    Make a target axis at 0,0 with ticks along the axis lines

    The top/right/left/bottom keywords toggle whether the corresponding plot border is drawn
    """
```

```

ax = axes or plt.gca()

ax.spines['right'].set_color('none')
ax.spines['top'].set_color('none')
ax.xaxis.set_ticks_position('bottom')
ax.spines['bottom'].set_position(('data',0))
ax.yaxis.set_ticks_position('left')
ax.spines['left'].set_position(('data',0))

#now re-enable visibles
if top:
    ax.xaxis.tick_top()
if bottom:
    ax.xaxis.tick_bottom()
if left:
    ax.yaxis.tick_left()
if right:
    ax.yaxis.tick_right()

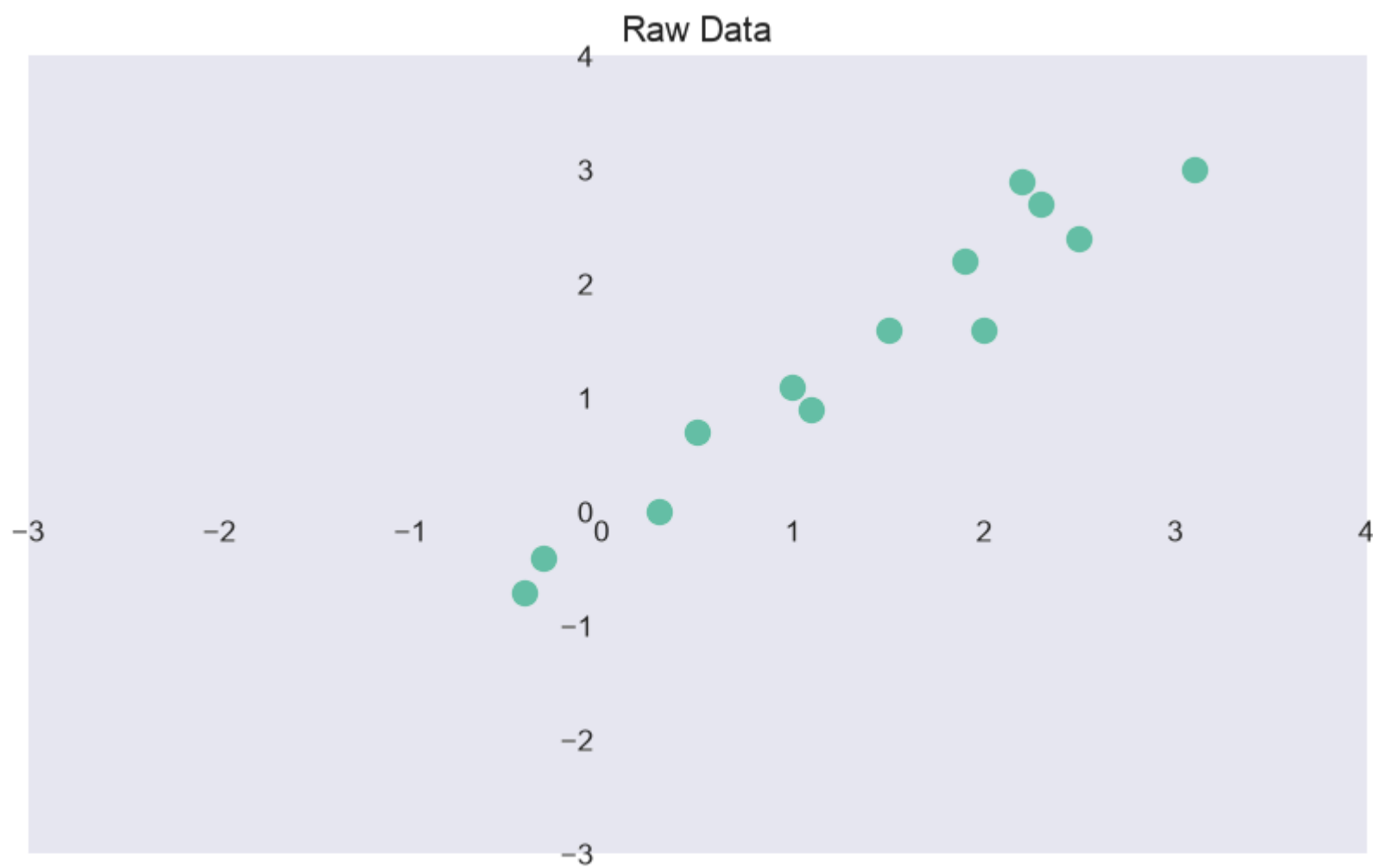
```

In [246...

```

import matplotlib.pyplot as plt
set2_colors = ['#66c2a5', '#fc8d62', '#8da0cb', '#e78ac3', '#a6d854', '#ffd92f', '#e5c494', '#b3b3b3']
ax = plt.gca()
ax.axis([-3, 4, -3, 4])
ax.scatter(df.x, df.y, s=100, c=set2_colors[0])
ax.set_title('Raw Data')
ax.grid(False)
cible_border()
plt.show()

```



Step 2: Subtract the mean (center the data)

```
In [247... mu = df.mean(axis=0)
print ("Mean:\n", mu, "\n")

df_0 = df - mu

ax = plt.gca()
ax.axis([-3, 4, -3, 4])
ax.scatter(df.x, df.y, s=100, c=set2_colors[0], label = "Raw Data")
ax.scatter(df_0.x, df_0.y, s=100, c=set2_colors[1], label = "Centered Data")
ax.set_title('Raw and mean-subtracted, centered data')
ax.grid(False)
ax.legend()
```

```
cible_border()
```

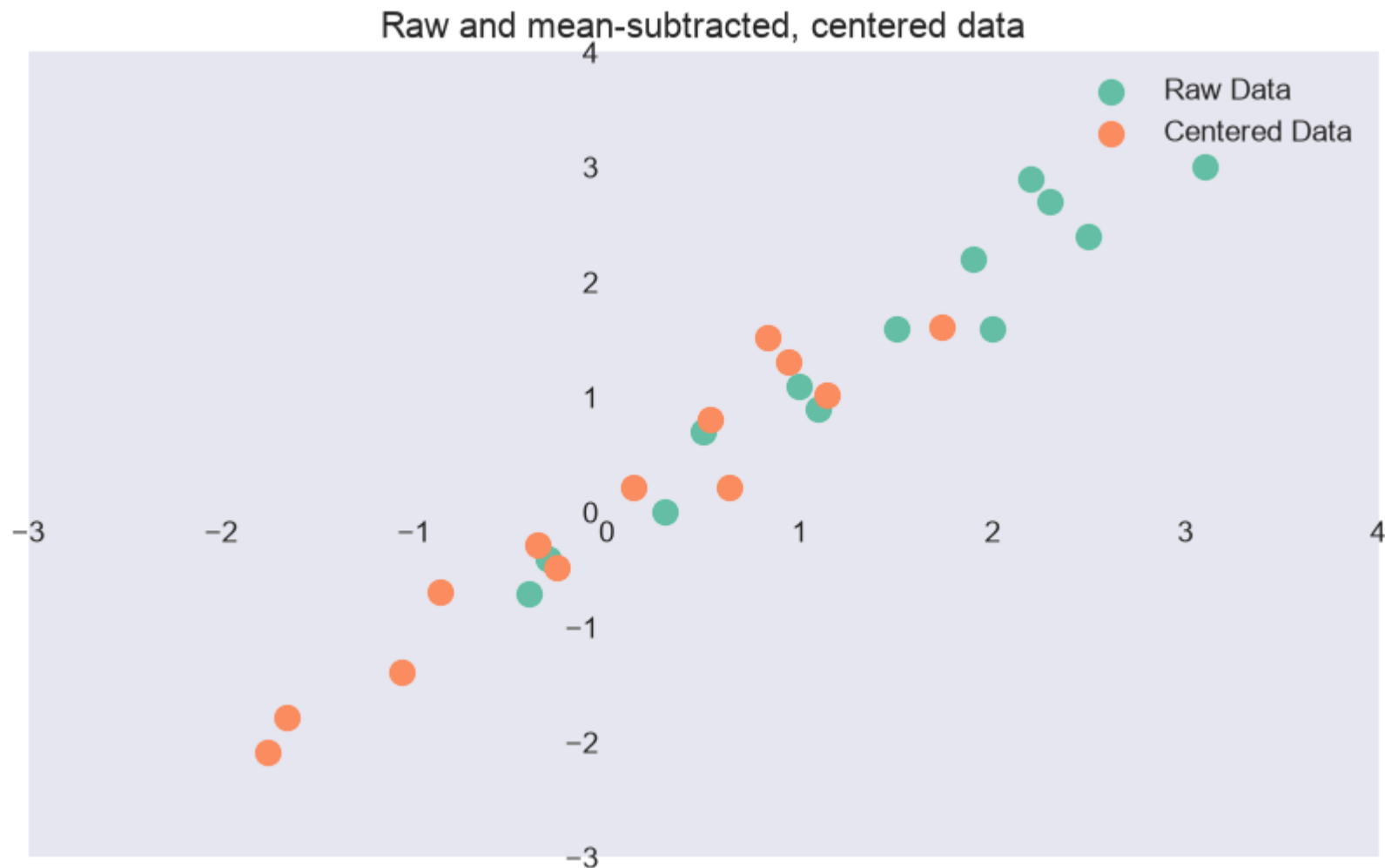
```
plt.show()
```

Mean:

x 1.361538

y 1.384615

dtype: float64



Step 3: Calculate the covariance matrix Σ

$$var(x) = \frac{\sum_{i=1}^n (X_i - \bar{X})(X_i - \bar{X})}{n-1}$$

(numpy.var default is divide by N)

$$\text{cov}(X, Y) = \frac{\sum_{i=1}^n (X_i - \bar{X})(Y_i - \bar{Y})}{n-1} \text{ (numpy.cov default is divide by N-1, sample covariance, unbiased estimate)}$$

$$\Sigma = \begin{pmatrix} \text{var}(x) & \text{cov}(x, y) \\ \text{cov}(x, y) & \text{var}(y) \end{pmatrix}$$

```
In [248... #D = np.cov(df.x, df.y) # all of the above gives the following output
#print(D)

#C = np.cov(df.T) #OR
#C = np.cov(df, rowvar=0) #OR
C = np.cov(df_0.x, df_0.y) # all of the above gives the following output
#C is a positive semi definite matrix (A positive semi definite matrix is a symmetric matrix with all positive >=0 eigenvalues)
print(C)
```

```
[[1.2125641  1.33352564]
 [1.33352564 1.55474359]]
```

- The diagonal (top left to bottom right) of covariance matrix is just variance $\text{cov}(x, x) \sim 1.212$, $\text{cov}(y, y) \sim 1.55$
- verify that with this command `np.var(df_0.x, ddof=1)`
- $\text{cov}(x, y) \sim 1.333$ = Since this is positive, both x and y are increasing/decreasing together. (This can be negative when x increases and y decreases or vice versa)
- Magnitude of covariance matrix is not interpretable (unlike correlation coefficient), since it depends on data.

Step 4: Calculate the eigenvector and eigenvalues of the covariance matrix

```
In [249... eigenvalues, eigenvectors = np.linalg.eigh(C)
print('Eigenvalues \n%s\n' %eigenvalues)
print('Eigenvectors \n%s' %eigenvectors)
```

```
Eigenvalues
[0.0391977  2.72810999]
```

```
Eigenvectors
[[-0.75075153  0.66058469]
 [ 0.66058469  0.75075153]]
```

The eigenvectors are all orthonormal

(Dot product is zero and vectors are unit length)

```
In [250... print(np.dot(eigenvectors[1], eigenvectors[0])) #Two vectors are orthogonal if the dot product of two vectors is 0
print('')
for vec in eigenvectors:
    print(np.round(np.linalg.norm(vec),6))
```

-5.5147102889211e-18

1.0

1.0

Sort the eigenvalues in descending order

- This gives you the components in order of significance.

```
In [251... idx = np.argsort(-eigenvalues )
eigenvalues = eigenvalues [ idx ]
eigenvectors = eigenvectors [: , idx ]
print(eigenvalues)
print(idx)
print('')
print(eigenvectors) ## columns also called phi1 , phi2, or PC1, PC2
print('')
```

[2.72810999 0.0391977]

[1 0]

[[0.66058469 -0.75075153]

[0.75075153 0.66058469]]

Check the the SVD decomposition

$$A = USV^T$$

`np.linalg.svd`

```
In [252... U = eigenvectors
S = eigenvalues*np.eye(2)
V = eigenvectors.T
A = np.dot(U, np.dot(S, V))

print(U, '\n')
print(S, '\n')
```

```
print(V, '\n')
print(A, '\n')
```

```
[[ 0.66058469 -0.75075153]
 [ 0.75075153  0.66058469]]
```

```
[[2.72810999 0.
 [0.          0.0391977 ]]
```

```
[[ 0.66058469  0.75075153]
 [-0.75075153  0.66058469]]
```

```
[[1.2125641  1.33352564]
 [1.33352564  1.55474359]]
```

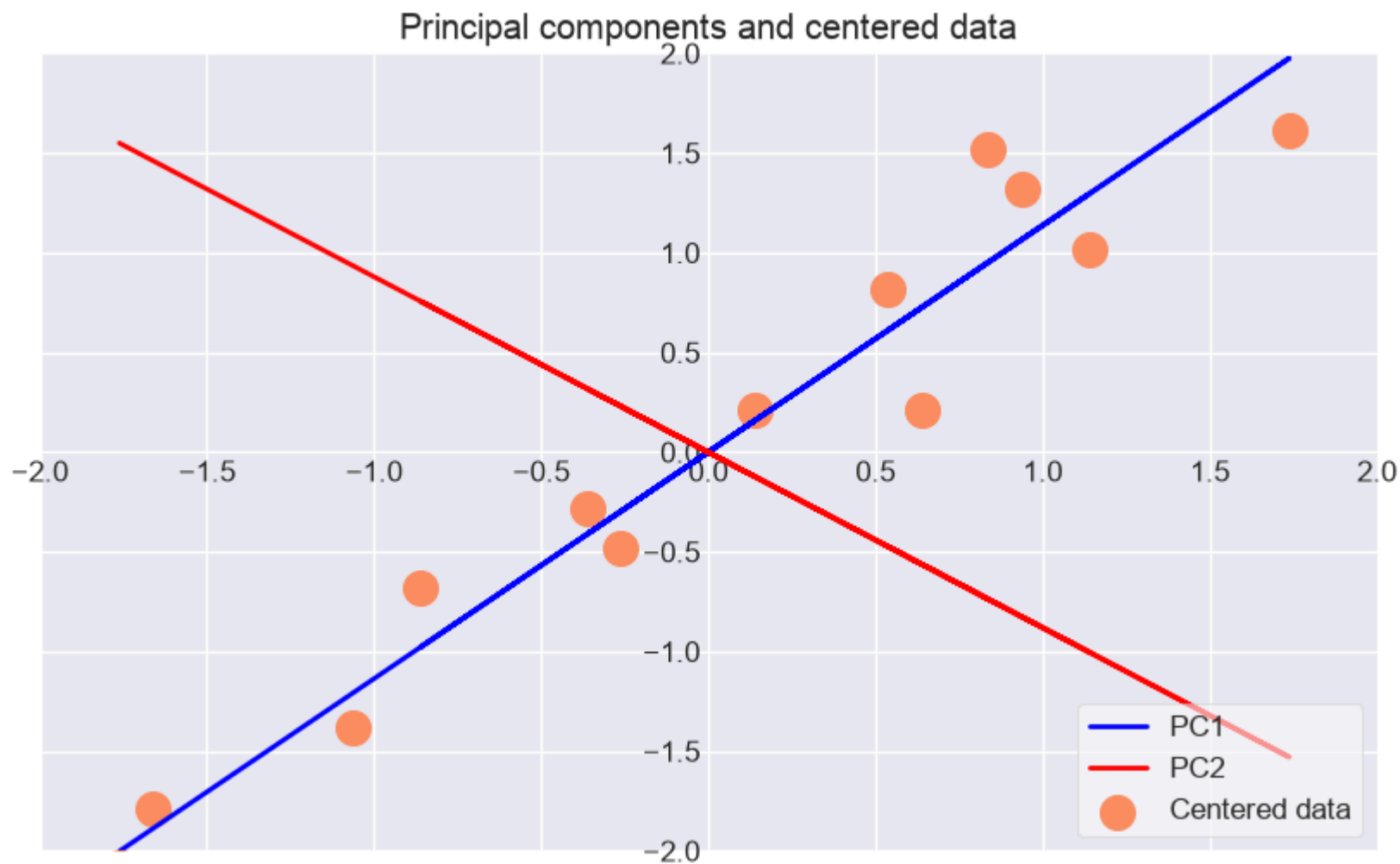
```
In [253... np.allclose(C,A) #comparing with covariance matrix
```

```
Out[253... True
```

Visualize the Principal Components

```
In [254... h1 = (eigenvectors[1][0]/eigenvectors[0][0])*df_0.x ##slope of the line
h2 = (eigenvectors[1][1]/eigenvectors[0][1])*df_0.x

ax = plt.gca()
ax.plot(df_0.x, h1, color='blue', label='PC1')
ax.plot(df_0.x, h2, color='red', label='PC2')
ax.axis([-2, 2, -2, 2])
ax.scatter(df_0.x, df_0.y, s=200, c=set2_colors[1], label='Centered data')
ax.set_title('Principal components and centered data')
ax.grid(True)
cible_border()
legend = ax.legend(loc=4, frameon=True, framealpha=0.6, scatterpoints=1)
rect = legend.get_frame()
rect.set_facecolor(light_grey)
rect.set_linewidth(0.6)
plt.show()
```



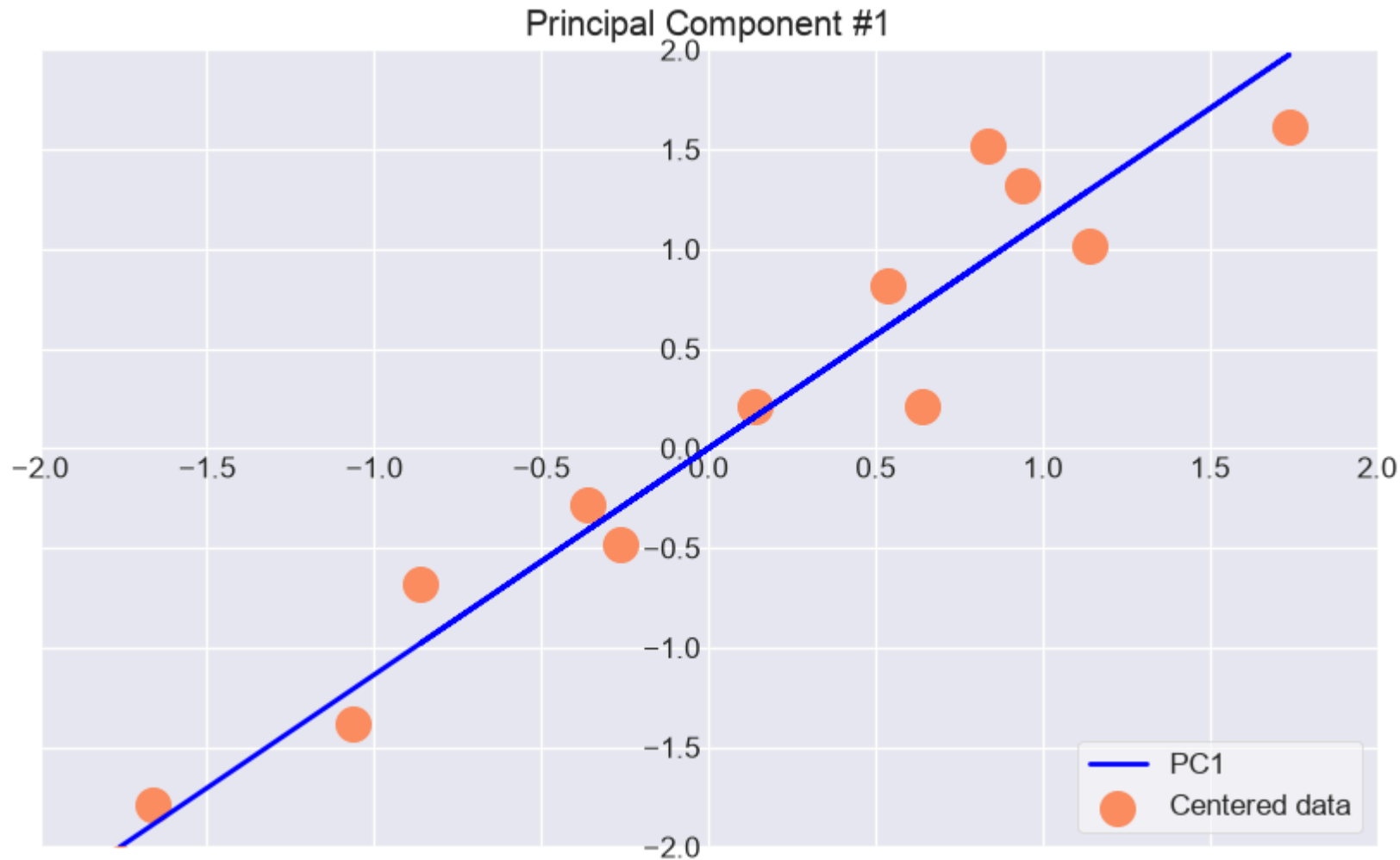
Step 5: Dimensionality Reduction - select the principal component(s)

Check the principal component

- Principal component: `eigenvalues[:,0]`

```
In [255... h = (eigenvalues[1][0]/eigenvalues[0][0])*df_0.x
ax = plt.gca()
ax.plot(df_0.x, h, color='blue', label='PC1')
ax.axis([-2, 2, -2, 2])
ax.scatter(df_0.x, df_0.y, s=200, c=set2_colors[1], label='Centered data')
ax.set_title('Principal Component #1')
ax.grid(True)
cible_border()
```

```
legend = ax.legend(loc=4, frameon=True, framealpha=0.6, scatterpoints=1)
rect = legend.get_frame()
rect.set_facecolor(light_grey)
rect.set_linewidth(0.6)
plt.show()
```



Select the significant component (principal component)

- The second component can be ignored if we choose to keep one significant component
- The final dataset will have reduced dimension (if the second component [or if anything beyond] is ignored.)
- Generally, some information could be lost, but may not be significant depending of the component.

- for d-dimensional original data, $k < d$ principal components can be selected, which reduces the dimension of the new data.
- select only the eigenvector(s) with higher eigenvalue
- Generally, $k < d$ eigenvectors with the highest k eigenvalues

Step 6: Projecting the data & Recovering the original data

- $Y = A \cdot F$
 - A: mean adjusted original data
 - F: Feature vector (Eigenvector)
 - Y: Transformed new data (Also called PC Scores)
- $OriginalData = Y \cdot F^T + mean$

See also:

<https://stats.stackexchange.com/questions/229092/how-to-reverse-pca-and-reconstruct-original-variables-from-several-principal-com>

$$PCA \text{ Reconstruction} = PC \text{ Scores} \cdot Eigenvectors^T + Mean$$

Step 6a: Projecting the data

In [258...

```
FeatureVector = eigenvectors
print("\n\nFeatureVector:\n", FeatureVector)

#
# TODO: Fill in the missing code to project the data points to the PCs
#
# FinalData = ???

FinalData = pd.DataFrame(FinalData)
print("\n\nFinalData (PC Scores):\n", FinalData)
```

```
FeatureVector:
[[ 0.66058469 -0.75075153]
 [ 0.75075153  0.66058469]]
```

```
FinalData (PC Scores):
      0      1
0 -2.728674 -0.054587
1 -2.437390  0.068513
2 -1.740738 -0.117704
3  1.514352 -0.183954
4 -1.083095  0.194555
5  1.691552  0.371564
6  0.967851  0.134380
7  2.361154 -0.238054
8  1.607460  0.164372
9  0.583458 -0.337046
10 -0.452502  0.083413
11  0.253166  0.038330
12 -0.536594 -0.123779
```

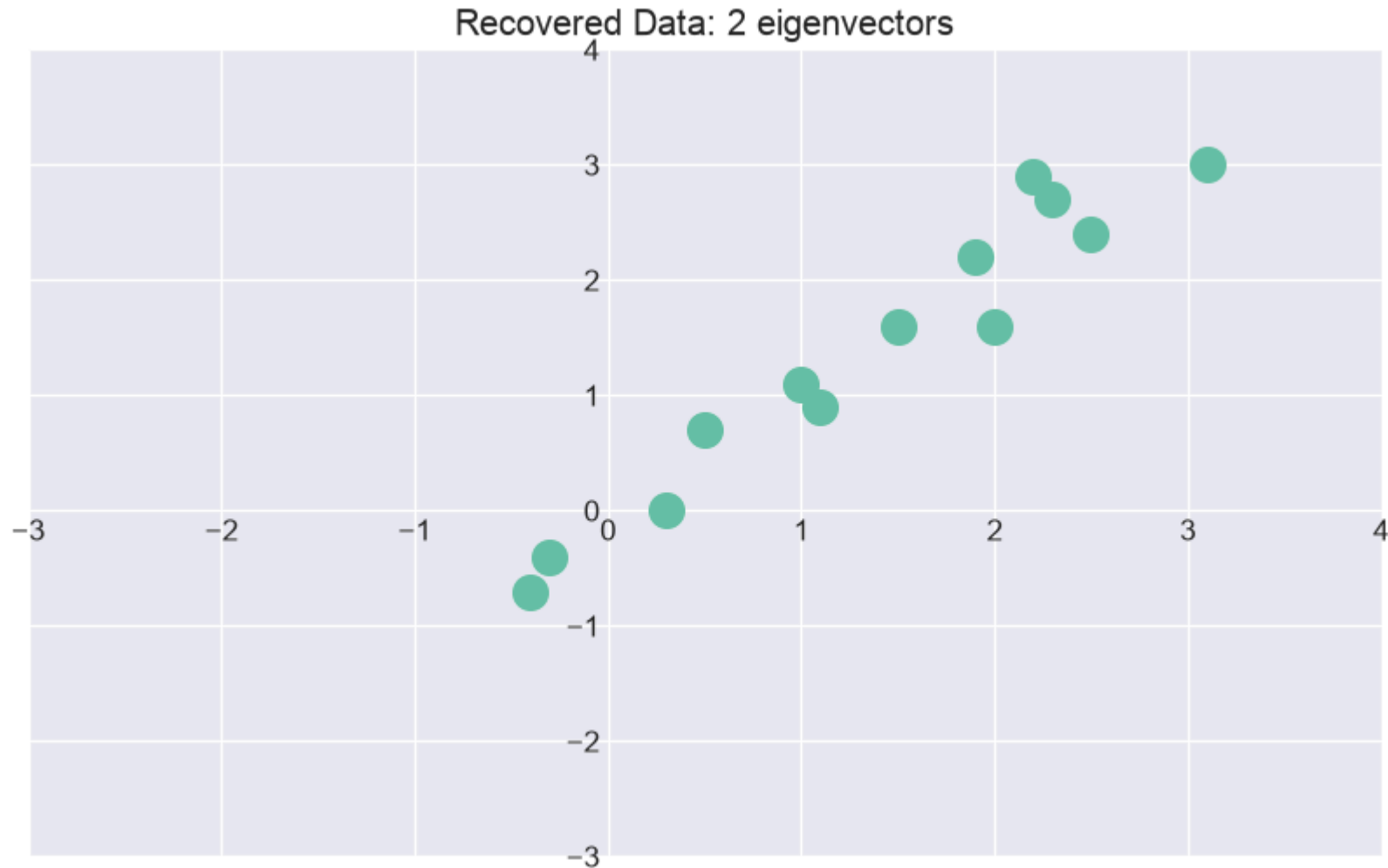
Step 6a: Recovering the data - 2 eigenvector case

```
In [259... #Getting data back
original = np.dot(np.array(FinalData),FeatureVector.T) + np.array(df.mean(axis=0))
print(original)
```

```
# As a reminder:
# x = [-0.4, -0.3, 0.3, 2.5,0.5, 2.2, 1.9, 3.1, 2.3, 2, 1, 1.5, 1.1]
# y = [-0.7,-0.4, 0., 2.4, 0.7, 2.9, 2.2,3.0,2.7,1.6,1.1,1.6,0.9]
```

```
[[-0.4 -0.7]
 [-0.3 -0.4]
 [ 0.3  0. ]
 [ 2.5  2.4]
 [ 0.5  0.7]
 [ 2.2  2.9]
 [ 1.9  2.2]
 [ 3.1  3. ]
 [ 2.3  2.7]
 [ 2.   1.6]
 [ 1.   1.1]
 [ 1.5  1.6]
 [ 1.1  0.9]]
```

```
In [260... ax = plt.gca()
ax.axis([-3, 4, -3, 4])
ax.scatter(original[:,0], original[:,1], s=200, c=set2_colors[0])
ax.set_title('Recovered Data: 2 eigenvectors')
ax.grid(True)
cible_border()
plt.show()
```



Step 7a: Reduced Dimension - Create a new data Y using the selected eigenvector(s)

- A case when only one component is selected.

```
In [265... #Projecting the Data

FeatureVector = eigenvectors[:,np.argsort(-eigenvalues)][:,0]
print("\n\nFeatureVector:\n",FeatureVector)

#
# TODO: Fill in the missing code to project the data points to the PCs
#
# FinalData = ???

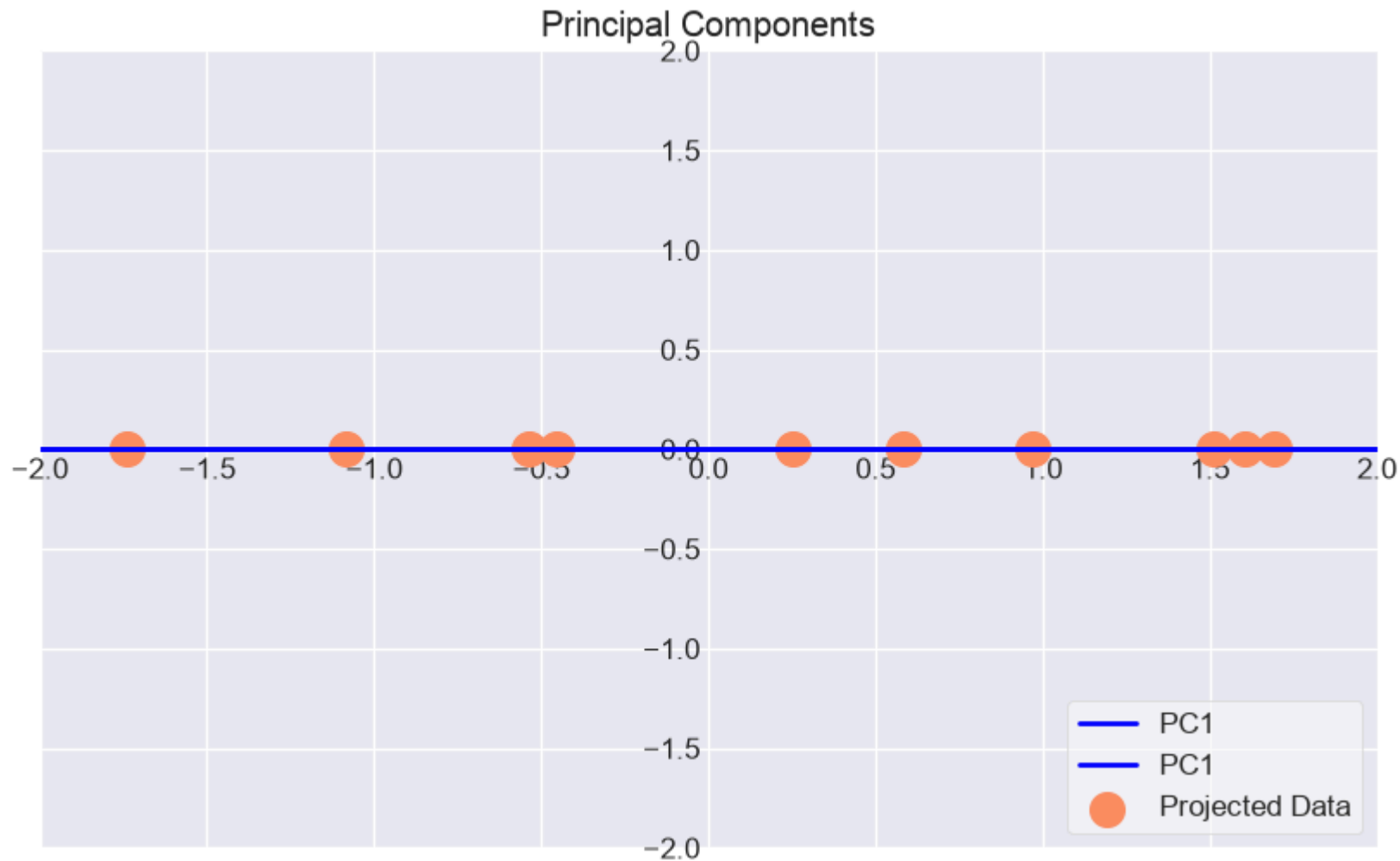
FinalData = pd.DataFrame(FinalData)
print("\n\nFinalData:\n",FinalData)
```

```
FeatureVector:
[0.66058469 0.75075153]
```

```
FinalData:
      0
0 -2.728674
1 -2.437390
2 -1.740738
3  1.514352
4 -1.083095
5  1.691552
6  0.967851
7  2.361154
8  1.607460
9  0.583458
10 -0.452502
11  0.253166
12 -0.536594
```

```
In [266... ax = plt.gca()
#ax.plot(df_0.x, np.dot(df_0, h), color='blue', label='PC1')
ax.axis([-2, 2, -2, 2])
ax.axhline(y=0, xmin=-2, xmax=2, color='blue', label='PC1')
ax.scatter(FinalData, np.zeros((len(FinalData))), s=200, c=set2_colors[1], label='Projected Data')
ax.set_title('Principal Components')
ax.grid(True)
cible_border()
legend = ax.legend(loc=4, frameon=True, framealpha=0.6, scatterpoints=1)
rect = legend.get_frame()
rect.set_facecolor(light_grey)
```

```
rect.set_linewidth(0.6)  
plt.show()
```



Step 7b: Recovering the original data

Recovering mean adjusted original data - for single eigenvector case

```
In [267... print(eigenvectors)  
df_0
```

```
[ [ 0.66058469 -0.75075153]  
  [ 0.75075153  0.66058469]]
```

Out [267...

	x	y
0	-1.761538	-2.084615
1	-1.661538	-1.784615
2	-1.061538	-1.384615
3	1.138462	1.015385
4	-0.861538	-0.684615
5	0.838462	1.515385
6	0.538462	0.815385
7	1.738462	1.615385
8	0.938462	1.315385
9	0.638462	0.215385
10	-0.361538	-0.284615
11	0.138462	0.215385
12	-0.261538	-0.484615

In [268... `V = eigenvectors[:,0]`

In [270... `#`
`# TODO: Fill in the missing code to project the data points to the PCs`
`#`
`# Y = ???`

`Y`

Out [270...

	0	1
0	-2.728674	-0.054587
1	-2.437390	0.068513
2	-1.740738	-0.117704
3	1.514352	-0.183954
4	-1.083095	0.194555
5	1.691552	0.371564
6	0.967851	0.134380
7	2.361154	-0.238054
8	1.607460	0.164372
9	0.583458	-0.337046
10	-0.452502	0.083413
11	0.253166	0.038330
12	-0.536594	-0.123779

In [273...

```
Y = Y.to_numpy().reshape(1, Y.size)
V = V.reshape(1, V.size)
A = np.dot(V.T, Y)
A = A.T
A
```

```
Out[273... array([[ -1.80251997, -2.04855584],
        [ -0.03605954, -0.04098151],
        [ -1.61010227, -1.82987398],
        [  0.0452586 ,  0.05143619],
        [ -1.149905 , -1.30686186],
        [ -0.07775353, -0.08836654],
        [  1.00035763,  1.13690195],
        [ -0.12151734, -0.1381039 ],
        [ -0.71547609, -0.81313536],
        [  0.12851997,  0.14606237],
        [  1.11741348,  1.26993539],
        [  0.24544922,  0.27895194],
        [  0.63934736,  0.7266154 ],
        [  0.08876922,  0.10088582],
        [  1.5597419 ,  1.77263965],
        [ -0.15725504, -0.17871964],
        [  1.0618637 ,  1.20680332],
        [  0.1085813 ,  0.12340216],
        [  0.38542359,  0.43803217],
        [ -0.22264756, -0.25303795],
        [ -0.29891603, -0.33971673],
        [  0.05510134,  0.06262243],
        [  0.16723752,  0.19006469],
        [  0.02531993,  0.02877598],
        [ -0.35446581, -0.4028488 ],
        [ -0.08176658, -0.09292735]])
```

```
In [274... ##### Recovering original data by putting back the subtracted mean
```

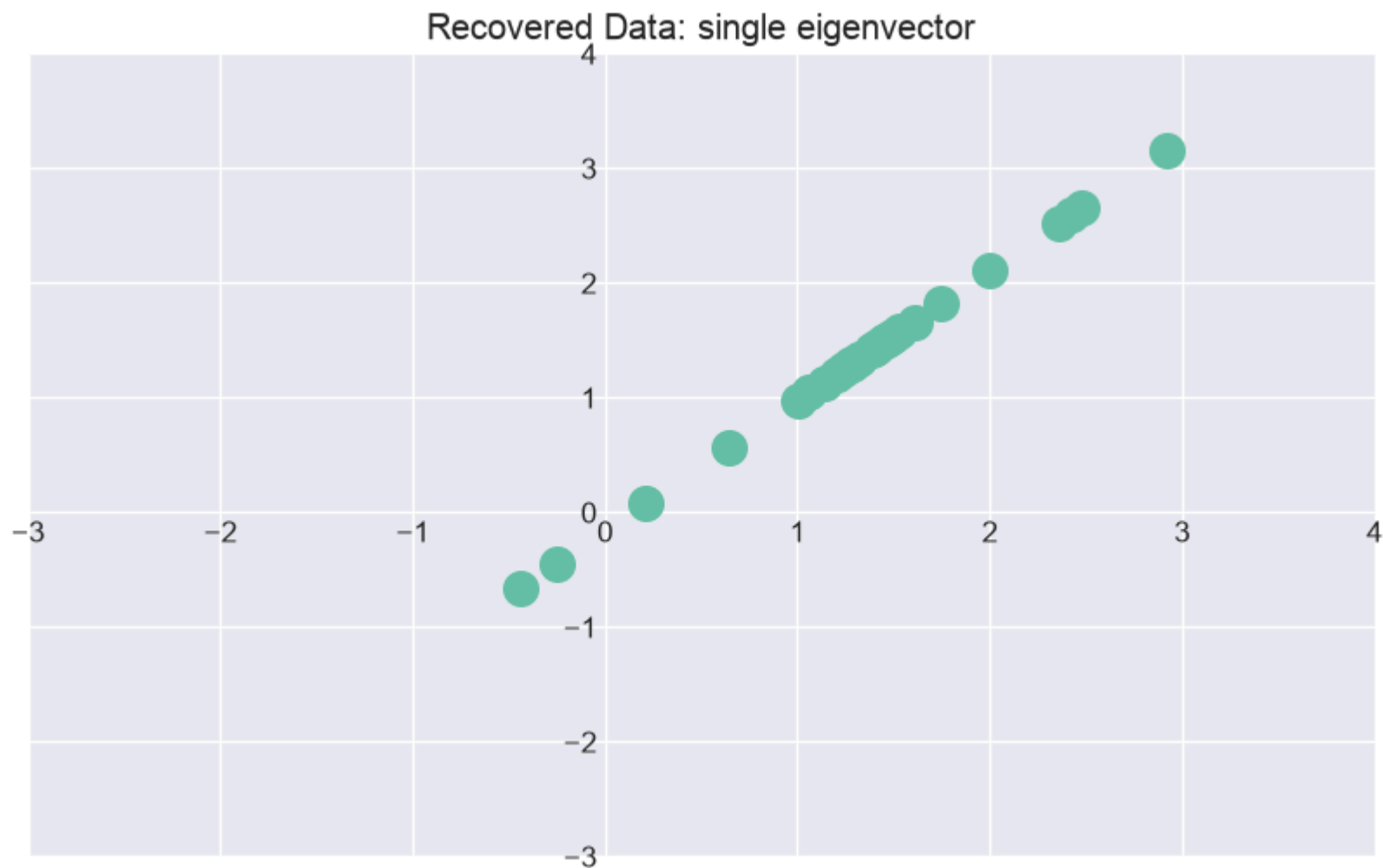
```
mu = df.mean(axis=0)
print (mu)
mu = mu.values.reshape(1, mu.size)
original = A + mu
```

```
x    1.361538
y    1.384615
dtype: float64
```

```
In [275... original
```

```
Out[275... array([[ -0.44098151, -0.66394046],
 [  1.32547892,  1.34363387],
 [-0.24856381, -0.4452586 ],
 [  1.40679706,  1.43605158],
 [  0.21163346,  0.07775353],
 [  1.28378493,  1.29624885],
 [  2.3618961 ,  2.52151734],
 [  1.24002113,  1.24651148],
 [  0.64606237,  0.57148003],
 [  1.49005843,  1.53067776],
 [  2.47895194,  2.65455078],
 [  1.60698769,  1.66356732],
 [  2.00088582,  2.11123078],
 [  1.45030768,  1.4855012 ],
 [  2.92128036,  3.15725504],
 [  1.20428343,  1.20589574],
 [  2.42340216,  2.5914187 ],
 [  1.47011976,  1.50801754],
 [  1.74696205,  1.82264756],
 [  1.1388909 ,  1.13157743],
 [  1.06262243,  1.04489866],
 [  1.4166398 ,  1.44723781],
 [  1.52877598,  1.57468007],
 [  1.38685839,  1.41339137],
 [  1.00707265,  0.98176658],
 [  1.27977188,  1.29168803]])
```

```
In [276... ax = plt.gca()
ax.axis([-3, 4, -3, 4])
plt.scatter(original[:,0], original[:,1], s=200, c=set2_colors[0])
ax.set_title('Recovered Data: single eigenvector')
ax.grid(True)
cible_border()
plt.show()
```



This plot does indeed show that we have lost some information compared to original, by selecting only one PC.

A look at real data - U.S. Arrests dataset

```
In [277... import sklearn
from sklearn import decomposition, preprocessing
from mpl_toolkits.mplot3d import Axes3D
import matplotlib as mpl
import matplotlib.pyplot as plt
```

```
In [278... #Originally this dataset is part of ISLR book/package in R – See chapter 10 in ISLR book
url = 'http://vincentarelbundock.github.io/Rdatasets/csv/datasets/USArrests.csv'
usarrests = pd.read_csv(url, sep=',')
# https://stackoverflow.com/questions/11346283/renaming-column-names-in-pandas
usarrests = usarrests.rename(columns={'rownames': 'State'})
usarrests.head()
```

```
Out[278...      State  Murder  Assault  UrbanPop  Rape
0  Alabama    13.2    236      58      21.2
1   Alaska    10.0    263      48      44.5
2  Arizona     8.1    294      80      31.0
3  Arkansas     8.8    190      50      19.5
4  California     9.0    276      91      40.6
```

```
In [279... usarrests_scaled = usarrests.copy()
cols = usarrests_scaled.columns[1:]
usarrests_scaled[cols] = preprocessing.StandardScaler().fit_transform(usarrests_scaled[cols])
usarrests_scaled.head()
```

```
Out[279...      State  Murder  Assault  UrbanPop  Rape
0  Alabama  1.255179  0.790787 -0.526195 -0.003451
1   Alaska  0.513019  1.118060 -1.224067  2.509424
2  Arizona  0.072361  1.493817  1.009122  1.053466
3  Arkansas  0.234708  0.233212 -1.084492 -0.186794
4  California  0.281093  1.275635  1.776781  2.088814
```

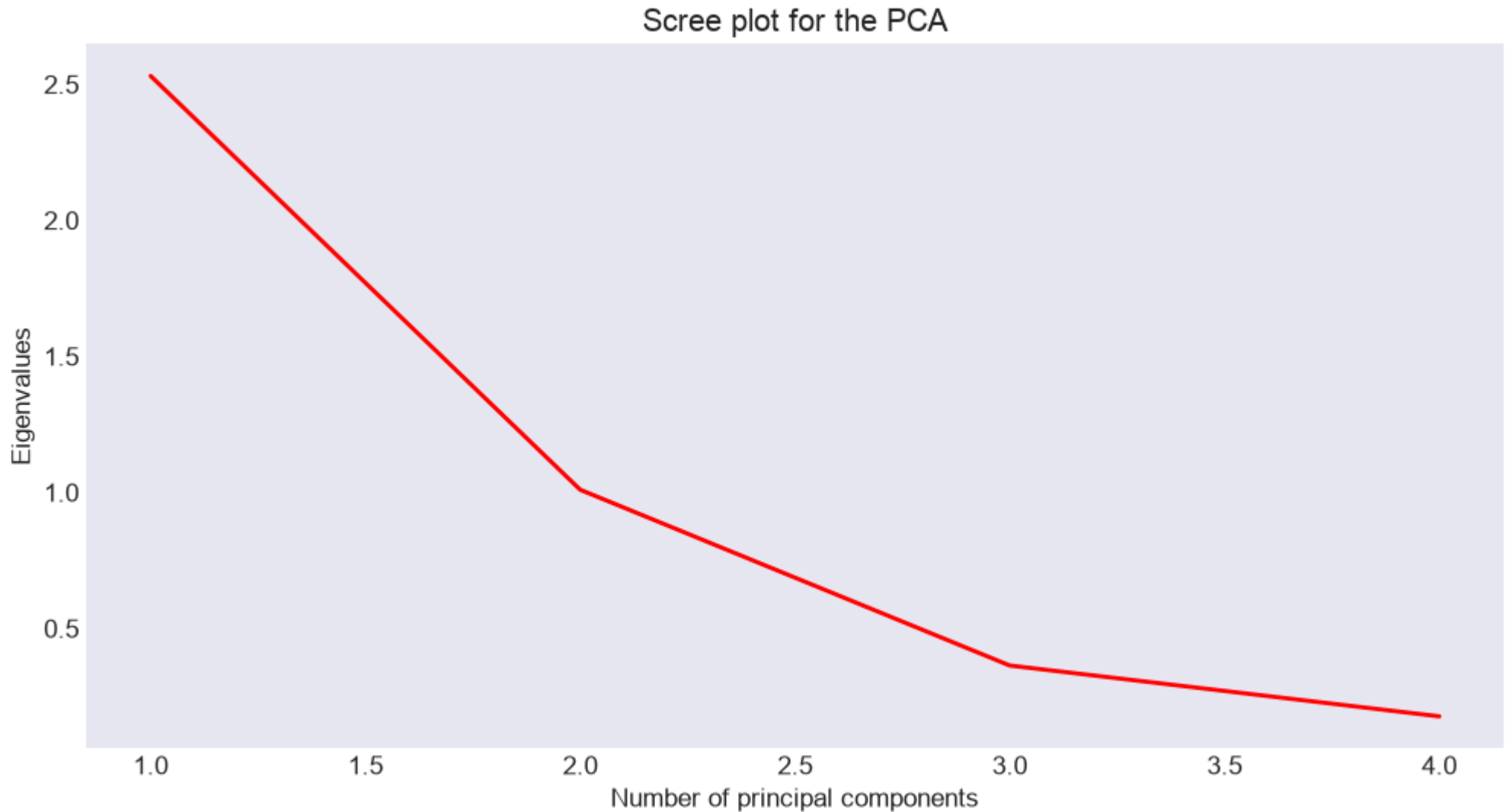
```
In [280... pca = sklearn.decomposition.PCA(n_components=4).fit(usarrests_scaled.iloc[:,1:])

coef_PCA = pca.transform(usarrests_scaled.iloc[:,1:]) #coef_PCA = projected Data/ PC scores
# we make a scree plot to see how many Principal Components to consider
plt.figure(figsize=(12, 6))
eig = pca.explained_variance_
# and calculate the variance explained by the PC analysis
var_exp = pca.explained_variance_ratio_.cumsum()*100.

plt.plot(np.arange(1, len(eig)+1), eig, color='r')
```

```
plt.title('Scree plot for the PCA')
plt.xlabel('Number of principal components')
plt.ylabel('Eigenvalues')
plt.show()

print ('The 1st Principal Component explains {:.03.1f} % of the variance\n'.format(var_exp[0]))
print ('The 1st and 2nd Principal Components explain {:.03.1f} % of the variance\n'.format(var_exp[1]))
print ('The 1st, 2nd and 3rd Principal Components explain {:.03.1f} % of the variance\n'.format(var_exp[2]))
print ('The first four Principal Components explain {:.03.1f} % of the variance\n'.format(var_exp[3]))
```



The 1st Principal Component explains 62.0 % of the variance

The 1st and 2nd Principal Components explain 86.8 % of the variance

The 1st, 2nd and 3rd Principal Components explain 95.7 % of the variance

The first four Principal Components explain 100.0 % of the variance

Results

The first two Principal Components explain almost 87% of the variation in the data. Since we can plot these two components in a single scatterplot, we have an effective way of showing the data concisely in a reduced dimension space - rather than 6 separate scatterplots for all combinations of the 4 original features.

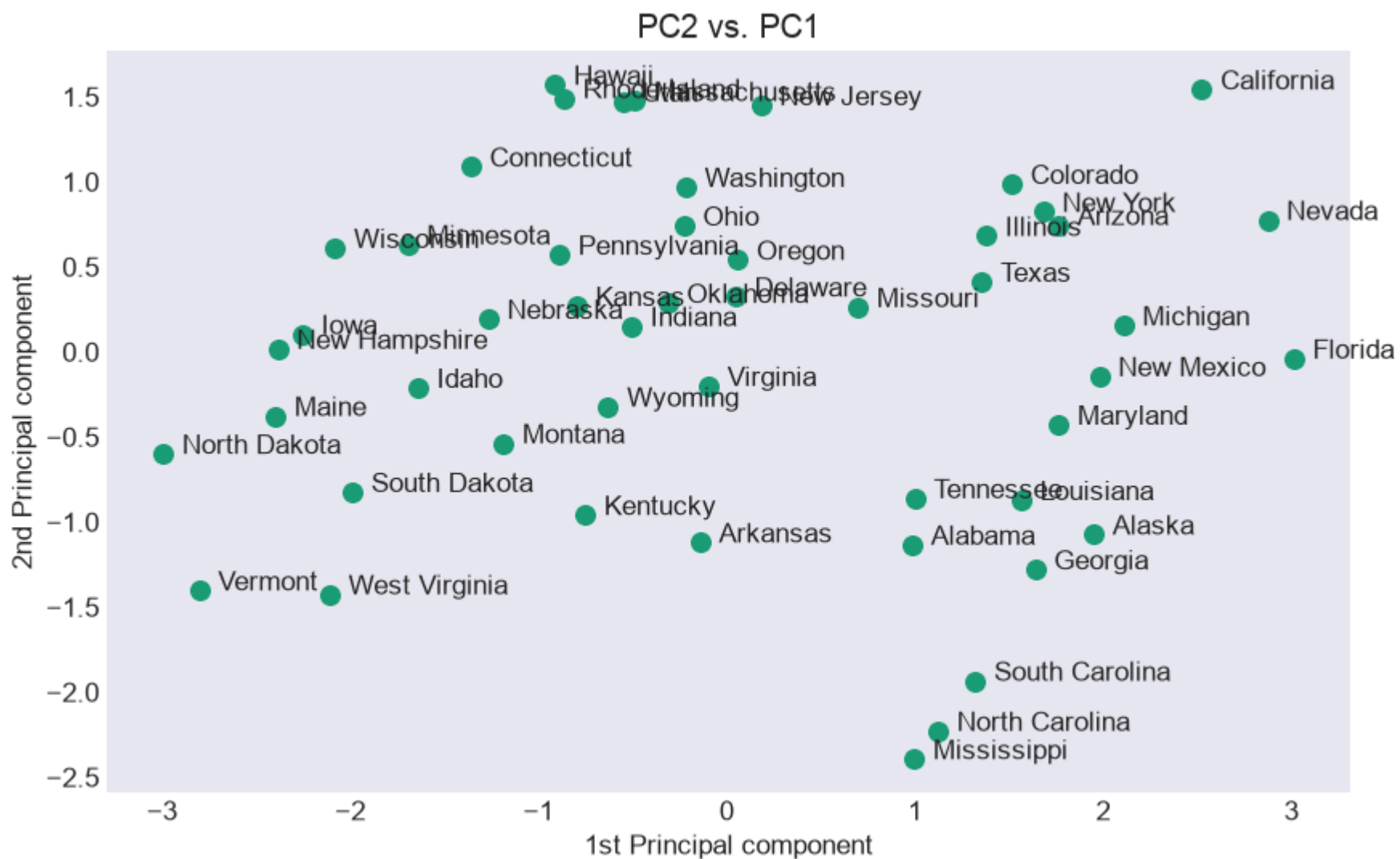
Principal Component score in a plot

Let's plot out the states in a scatterplot according to the coefficients of the PC scores

```
In [281...] userarrests_scaled.columns
```

```
Out[281...] Index(['State', 'Murder', 'Assault', 'UrbanPop', 'Rape'], dtype='str')
```

```
In [282...] dark2_colors = ['#1b9e77', '#d95f02', '#7570b3', '#e7298a', '#66a61e', '#e6ab02', '#a6761d', '#666666']
plt.figure(figsize=(10,6))
plt.scatter(coef_PCA[:,0], coef_PCA[:,1], s = 70, zorder=0, c=dark2_colors[0])
for i in range(coef_PCA.shape[0]):
    plt.text(coef_PCA[i,0]+.1, coef_PCA[i,1], userarrests_scaled.iloc[i]['State'], zorder=1, size=12, color=shade_black)
plt.ylabel('2nd Principal component' )
plt.xlabel('1st Principal component' )
plt.title('PC2 vs. PC1')
plt.show()
```



In [283...] `pca.components_`

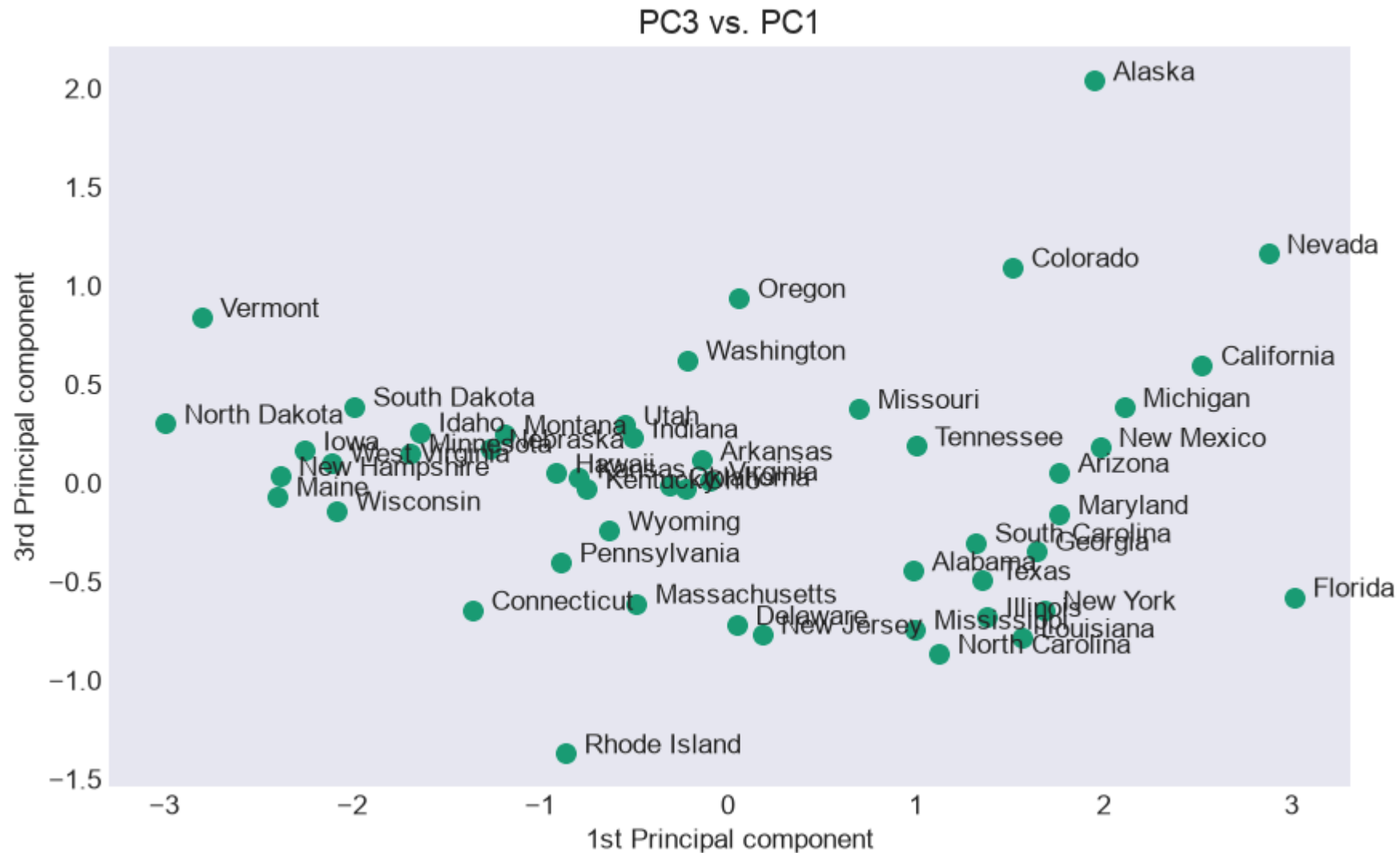
Out[283...] `array([[0.53589947, 0.58318363, 0.27819087, 0.54343209],
 [-0.41818087, -0.1879856 , 0.87280619, 0.16731864],
 [-0.34123273, -0.26814843, -0.37801579, 0.81777791],
 [-0.6492278 , 0.74340748, -0.13387773, -0.08902432]])`

Looking at the plot of PC2 against PC1, what is your interpretation of the axes?

(The state names represent the scores for the first two principal components. - See book for Biplot and interpretation) `coef_PCA` #these are called scores #Each row corresponds to each state

Next, let's plot the third Principal Component against the first

```
In [284... plt.figure(figsize=(10,6))
plt.scatter(coef_PCA[:,0], coef_PCA[:,2], s = 70, zorder=0, c=dark2_colors[0])
for i in range(coef_PCA.shape[0]):
    plt.text(coef_PCA[i,0]+.1, coef_PCA[i,2], usarrests_scaled.iloc[i]['State'], zorder=1, size=12, color=shade_black)
plt.ylabel('3rd Principal component' )
plt.xlabel('1st Principal component' )
plt.title('PC3 vs. PC1')
plt.show()
```



We can also plot all three of the first Principal Components together in a 3D scatterplot

In [285...

```
%matplotlib inline
# https://matplotlib.org/stable/gallery/mplot3d/scatter3d.html

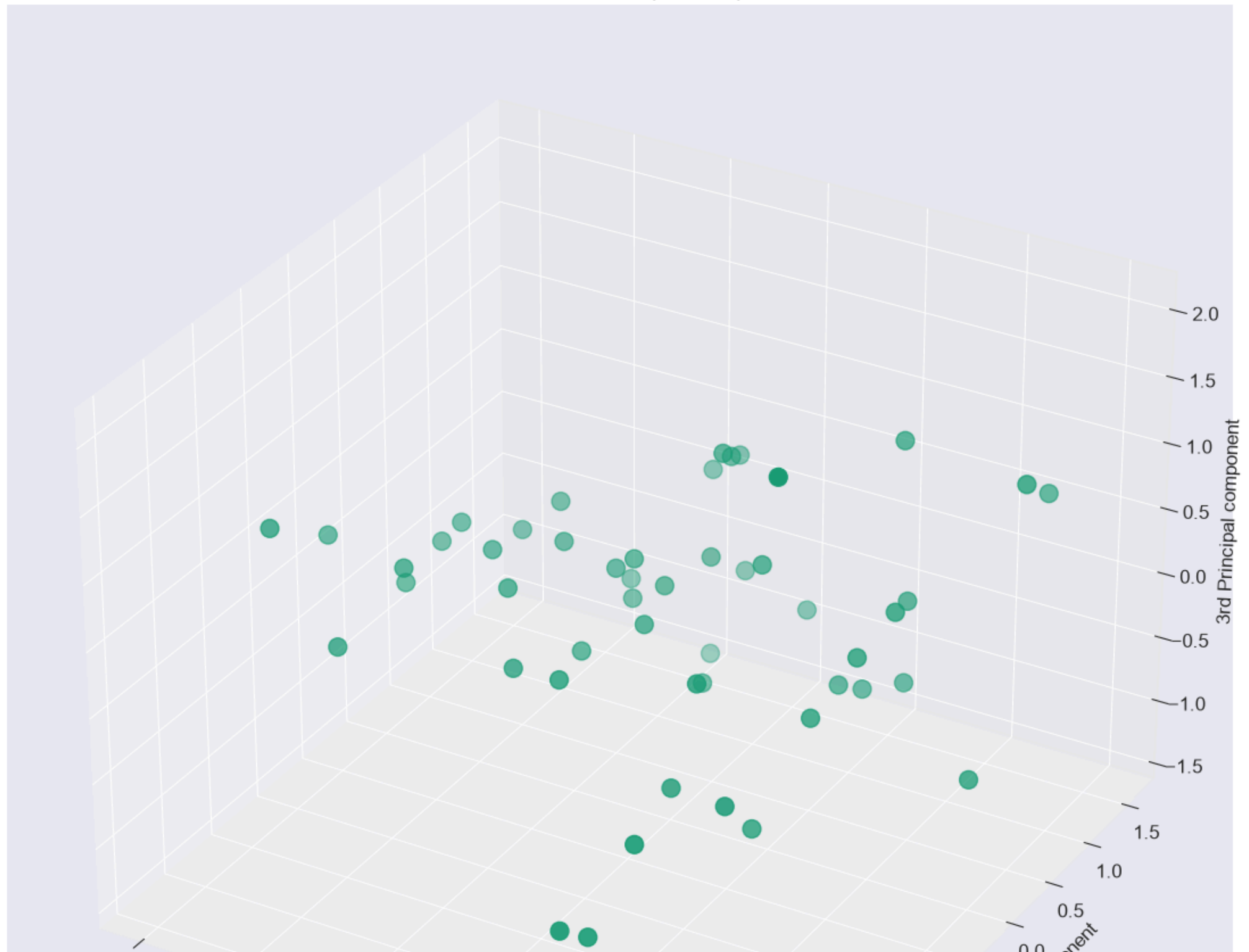
fig = plt.figure(1, figsize=(14, 14))
ax = fig.add_subplot(projection='3d')

ax.scatter(coef_PCA[:,0], coef_PCA[:,1], coef_PCA[:,2], s = 130, zorder=0, c=dark2_colors[0])

#for i in range(coef_PCA.shape[0]):
#    ax.text(coef_PCA[i,0], coef_PCA[i,1], coef_PCA[i,2], usarrests_scaled.iloc[i]['State'], zorder=1, size=12, color=shade_bla

ax.set_xlabel('1st Principal component')
ax.set_ylabel('2nd Principal component')
ax.set_zlabel('3rd Principal component')
ax.set_title('1st, 2nd and 3rd Principal Components')
plt.draw();
```

1st, 2nd and 3rd Principal Components





```
In [286... print([s for s in mpl.style.available if 'seaborn' in s])
```

```
['seaborn-v0_8', 'seaborn-v0_8-bright', 'seaborn-v0_8-colorblind', 'seaborn-v0_8-dark', 'seaborn-v0_8-dark-palette', 'seaborn-v0_8-darkgrid', 'seaborn-v0_8-deep', 'seaborn-v0_8-muted', 'seaborn-v0_8-notebook', 'seaborn-v0_8-paper', 'seaborn-v0_8-pastel', 'seaborn-v0_8-poster', 'seaborn-v0_8-talk', 'seaborn-v0_8-ticks', 'seaborn-v0_8-white', 'seaborn-v0_8-whitegrid']
```

```
In [287... %matplotlib inline
mpl.rcParams.update(mpl.rcParamsDefault)
set_mpl_params()
mpl.style.use('seaborn-v0_8-dark')
```

Feature loading plot

We can also plot the principal component loading vector coefficient for each of the features. This allows us to make an interpretation of the Principal Components on the basis of the dispersion of the feature loadings.

This is the a way of showing graphically the Correlation Circle that was referred to in lecture. ((See <http://www.sthda.com/english/articles/31-principal-component-methods-in-r-practical-guide/112-pca-principal-component-analysis-essentials/> for correlation circle interpretation)

```
In [292... def make_loading_plot(components, cols):
    summ_up = lambda x, y: abs(x) + abs(y)
    loadings = pd.DataFrame(data=dict(X=components[0], Y=components[1], name=cols))
    #loadings['summ'] = loadings.apply(lambda row: summ_up(row['X'], row['Y']), axis=1)
    plt.scatter(loadings.X, loadings.Y, s=200, label="Features",
                facecolor=dark2_colors[1], edgecolor=shade_black, linewidth=0.35);

    for i in range(loadings.shape[0]):
        plt.text(loadings.iloc[i]['X']+.01, loadings.iloc[i]['Y'], loadings.iloc[i]['name'], size=12, color=shade_black)

    plt.title("Feature loadings for the PCA");
```

```
plt.xlabel("Feature loadings on PC1");
plt.ylabel("Feature loadings on PC2");
plt.grid(True)

legend = plt.legend(loc=2, frameon=True, framealpha=0.6, scatterpoints=1)
rect = legend.get_frame()
rect.set_facecolor(light_grey)
rect.set_linewidth(0.6)
```

```
make_loading_plot(pca.components_, usarrests_scaled.columns[1:])
```

```
plt.show();
```



```
In [293... pd.DataFrame(pca.components_, columns=usarrests_scaled.columns[1:], index = ['PC1', 'PC2', 'PC3', 'PC4']).T
```

```
Out[293... 
```

	PC1	PC2	PC3	PC4
Murder	0.535899	-0.418181	-0.341233	-0.649228
Assault	0.583184	-0.187986	-0.268148	0.743407
UrbanPop	0.278191	0.872806	-0.378016	-0.133878
Rape	0.543432	0.167319	0.817778	-0.089024

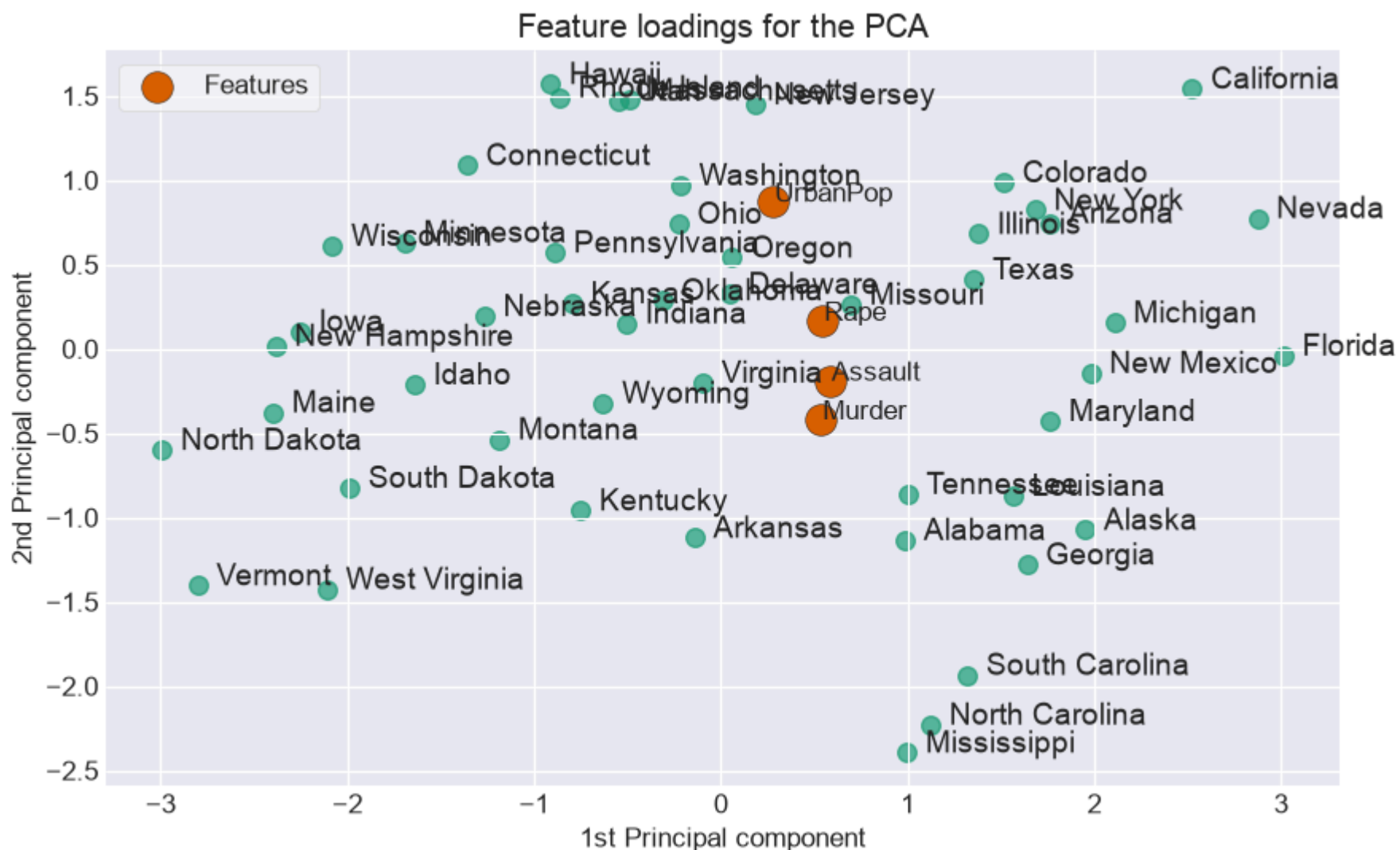
Interpretation of above plot

We see that the violent crimes have roughly equal coefficients on the (positive) end of PC1. They are also close together, which indicates correlation with each other. In contrast, UrbanPoP is much less. PC2 seems to oppose UrbanPop and the violent crime features - representing the degree of urbanization of the state. Note also that Rape and Murder are also separated by this PC.

Let's look at both plots together, and interpret

(This is similar to biplot in R)

```
In [294... make_loading_plot(pca.components_, usarrests_scaled.columns[1:])
plt.scatter(coef_PCA[:,0], coef_PCA[:,1], s = 70, alpha=0.7, zorder=0, color = dark2_colors[0])
for i in range(coef_PCA.shape[0]):
    plt.text(coef_PCA[i,0]+.1, coef_PCA[i,1], usarrests_scaled.iloc[i]['State'], zorder=1, size=14, color=shade_black)
plt.ylabel('2nd Principal component' )
plt.xlabel('1st Principal component' )
plt.show()
```



The first two principal components for the USArrests data. The blue state names represent the scores for the first two principal components. The orange arrows indicate the first two principal component loading vectors. This plot is almost similar to biplot in R. It's called biplot because it displays both loadings and principal component scores.

[back to top](#)

USArrests Dataset (No sklearn)

```
In [296... #Originally this dataset is part of ISLR book/package in R – See chapter 10 in ISLR book
url = 'http://vincentarelbundock.github.io/Rdatasets/csv/datasets/USArrests.csv'
usarrests = pd.read_csv(url, sep=',')
```

```

#usarrests.columns = usarrests.columns.str.replace('Unnamed: 0', 'State')
usarrests = usarrests.rename(columns={'rownames': 'State'})
print(usarrests.head())

df1 = usarrests.iloc[:,1:] - usarrests.iloc[:,1:].mean(axis=0)
print("\n\nMean centered")
print(df1.head())

df1 = df1/usarrests.iloc[:,1:].std()
print("\n\nStandardized")
print(df1.head())

cov_df1 = np.cov(df1.T)
print("\n\ncovariance") #or correlation because we standardized
np.set_printoptions(suppress=True)
print(np.round(cov_df1,6))

eigenvalues,eigenvectors = np.linalg.eigh(cov_df1)
print("\n\nEigenvalues")
print(eigenvalues)
print("\n\nEigenvectors:\n",eigenvectors)

#Getting data back

FeatureVector = eigenvectors[:,np.argsort(-eigenvalues)]
print("\n\nFeatureVector:\n",FeatureVector)

#
# TODO: Fill in the missing code to project the data points to the PCs
#
# pcscores = ???

np.dot(pcscores,FeatureVector.T) * np.array(usarrests.iloc[:,1:].std()).reshape(4,1).T + np.array(usarrests.iloc[:,1:].mean(

```

	State	Murder	Assault	UrbanPop	Rape
0	Alabama	13.2	236	58	21.2
1	Alaska	10.0	263	48	44.5
2	Arizona	8.1	294	80	31.0
3	Arkansas	8.8	190	50	19.5
4	California	9.0	276	91	40.6

Mean centered

	Murder	Assault	UrbanPop	Rape
0	5.412	65.24	-7.54	-0.032
1	2.212	92.24	-17.54	23.268
2	0.312	123.24	14.46	9.768
3	1.012	19.24	-15.54	-1.732
4	1.212	105.24	25.46	19.368

Standardized

	Murder	Assault	UrbanPop	Rape
0	1.242564	0.782839	-0.520907	-0.003416
1	0.507862	1.106823	-1.211764	2.484203
2	0.071633	1.478803	0.998980	1.042878
3	0.232349	0.230868	-1.073593	-0.184917
4	0.278268	1.262814	1.758923	2.067820

covariance

```
[[1.      0.801873 0.069573 0.563579]
 [0.801873 1.      0.258872 0.665241]
 [0.069573 0.258872 1.      0.411341]
 [0.563579 0.665241 0.411341 1.      ]]
```

Eigenvalues

```
[0.17343009 0.35656318 0.98976515 2.48024158]
```

Eigenvectors:

```
[[ 0.6492278 -0.34123273 0.41818087 -0.53589947]
 [-0.74340748 -0.26814843 0.1879856 -0.58318363]
 [ 0.13387773 -0.37801579 -0.87280619 -0.27819087]
 [ 0.08902432 0.81777791 -0.16731864 -0.54343209]]
```

FeatureVector:

```
[[-0.53589947 0.41818087 -0.34123273 0.6492278 ]]
```

```
[-0.58318363  0.1879856 -0.26814843 -0.74340748]  
[-0.27819087 -0.87280619 -0.37801579  0.13387773]  
[-0.54343209 -0.16731864  0.81777791  0.08902432]]
```

```
Out[296... array([[ 13.2, 236. , 58. , 21.2],
 [ 10. , 263. , 48. , 44.5],
 [  8.1, 294. , 80. , 31. ],
 [  8.8, 190. , 50. , 19.5],
 [  9. , 276. , 91. , 40.6],
 [  7.9, 204. , 78. , 38.7],
 [  3.3, 110. , 77. , 11.1],
 [  5.9, 238. , 72. , 15.8],
 [ 15.4, 335. , 80. , 31.9],
 [ 17.4, 211. , 60. , 25.8],
 [  5.3,  46. , 83. , 20.2],
 [  2.6, 120. , 54. , 14.2],
 [ 10.4, 249. , 83. , 24. ],
 [  7.2, 113. , 65. , 21. ],
 [  2.2,  56. , 57. , 11.3],
 [  6. , 115. , 66. , 18. ],
 [  9.7, 109. , 52. , 16.3],
 [ 15.4, 249. , 66. , 22.2],
 [  2.1,  83. , 51. ,  7.8],
 [ 11.3, 300. , 67. , 27.8],
 [  4.4, 149. , 85. , 16.3],
 [ 12.1, 255. , 74. , 35.1],
 [  2.7,  72. , 66. , 14.9],
 [ 16.1, 259. , 44. , 17.1],
 [  9. , 178. , 70. , 28.2],
 [  6. , 109. , 53. , 16.4],
 [  4.3, 102. , 62. , 16.5],
 [ 12.2, 252. , 81. , 46. ],
 [  2.1,  57. , 56. ,  9.5],
 [  7.4, 159. , 89. , 18.8],
 [ 11.4, 285. , 70. , 32.1],
 [ 11.1, 254. , 86. , 26.1],
 [ 13. , 337. , 45. , 16.1],
 [  0.8,  45. , 44. ,  7.3],
 [  7.3, 120. , 75. , 21.4],
 [  6.6, 151. , 68. , 20. ],
 [  4.9, 159. , 67. , 29.3],
 [  6.3, 106. , 72. , 14.9],
 [  3.4, 174. , 87. ,  8.3],
 [ 14.4, 279. , 48. , 22.5],
 [  3.8,  86. , 45. , 12.8],
 [ 13.2, 188. , 59. , 26.9],
 [ 12.7, 201. , 80. , 25.5],
 [  3.2, 120. , 80. , 22.9],
 [  2.2,  48. , 32. , 11.2],
 [  8.5, 156. , 63. , 20.7],
```

```
[ 4. , 145. , 73. , 26.2],  
[ 5.7, 81. , 39. , 9.3],  
[ 2.6, 53. , 66. , 10.8],  
[ 6.8, 161. , 60. , 15.6]])
```

[back to top](#)

Application of PCA for image manipulation

This part of the Section and notebook, is extra, if you are interested in a an application of PCA to images. It doesn't contain any information that is essential for the Homework, and feel free to skip this.

Eigenfaces

- AT&T faces
- The database for the faces can be found in link below
- <http://cam-orl.co.uk/facedatabase.html/>

This dataset contains a set of face images taken between April 1992 and April 1994 at AT&T Laboratories Cambridge. The `sklearn.datasets.fetch_olivetti_faces` function is the data fetching / caching function that downloads the data archive from AT&T.

If you want to read the images from the files and populate the data into a matrix X as a row data

- The images array has m x n image data
- The data array has the m x n images in a row (flattened)
- The target array holds the subject's id, ranging from 0 to 39
- Each row holds one image data

```
In [78]: from sklearn import datasets  
faces = sklearn.datasets.fetch_olivetti_faces()
```

downloading Olivetti faces from <https://ndownloader.figshare.com/files/5976027> to /Users/similiabhilash/scikit_learn_data

```
In [79]: print('Shape of target array: ', faces.target.shape, '\n' )  
print('Shape of images array: ', faces.images.shape, '\n' )  
print('Shape of data array: ', faces.data.shape)
```

Shape of target array: (400,)

Shape of images array: (400, 64, 64)

Shape of data array: (400, 4096)

In [80]: *### Examine original image using the function below.*

```
def view_original(image):  
    plt.figure(figsize=(4,4))  
    plt.gray()  
    plt.axis('off')  
    plt.imshow(image)  
    return np.array(image).shape[0:2]  
  
m,n = view_original(faces.images[30,:])  
plt.show()  
print("shape of image is", m,n)
```



shape of image is 64 64

In [81]: `I = faces.data`
`print(I.shape)`

(400, 4096)

Perform PCA calculation

Question:

*How many Principal Components can I have, maximum? *

Answer: 400 - because in this example we are treating the individual images as features, and the 4096 points as the data samples. We can have a maximum of principal components equal to the number of features - i.e. the number of original images.

```
In [82]: def pca_calculation(I, num_components):
    [nv, d] = I.shape
    print(nv,d)
    mean_I = np.mean(I, 0)
    ma_I = I - mean_I
    # calculate the eigenvectors (eigenimages) and eigenvalues
    M = np.dot(ma_I,ma_I.T) #covariance matrix
    eigenvalues,eigenvectors = np.linalg.eigh(M)
    eigenvectors = np.dot(ma_I.T, eigenvectors)
    for i in range (nv):
        eigenvectors[:,i] = eigenvectors[:,i]/ np.linalg.norm ( eigenvectors[:, i])
    idx = np.argsort(-eigenvalues) #returns the index with increasing order
        #(-z1 makes it decreasing order)

    eigenvalues = eigenvalues[idx]
    eigenvectors = eigenvectors[:, idx]

    print(M.shape[0:2])
    #return only num_components
    eigenvalues = eigenvalues[0: num_components].copy()
    eigenvectors = eigenvectors[:,0: num_components ].copy()
    return(eigenvalues, eigenvectors, mean_I)

num_components = 8
eigenvalues, eigenvectors, mean_I = pca_calculation(I, num_components)
```

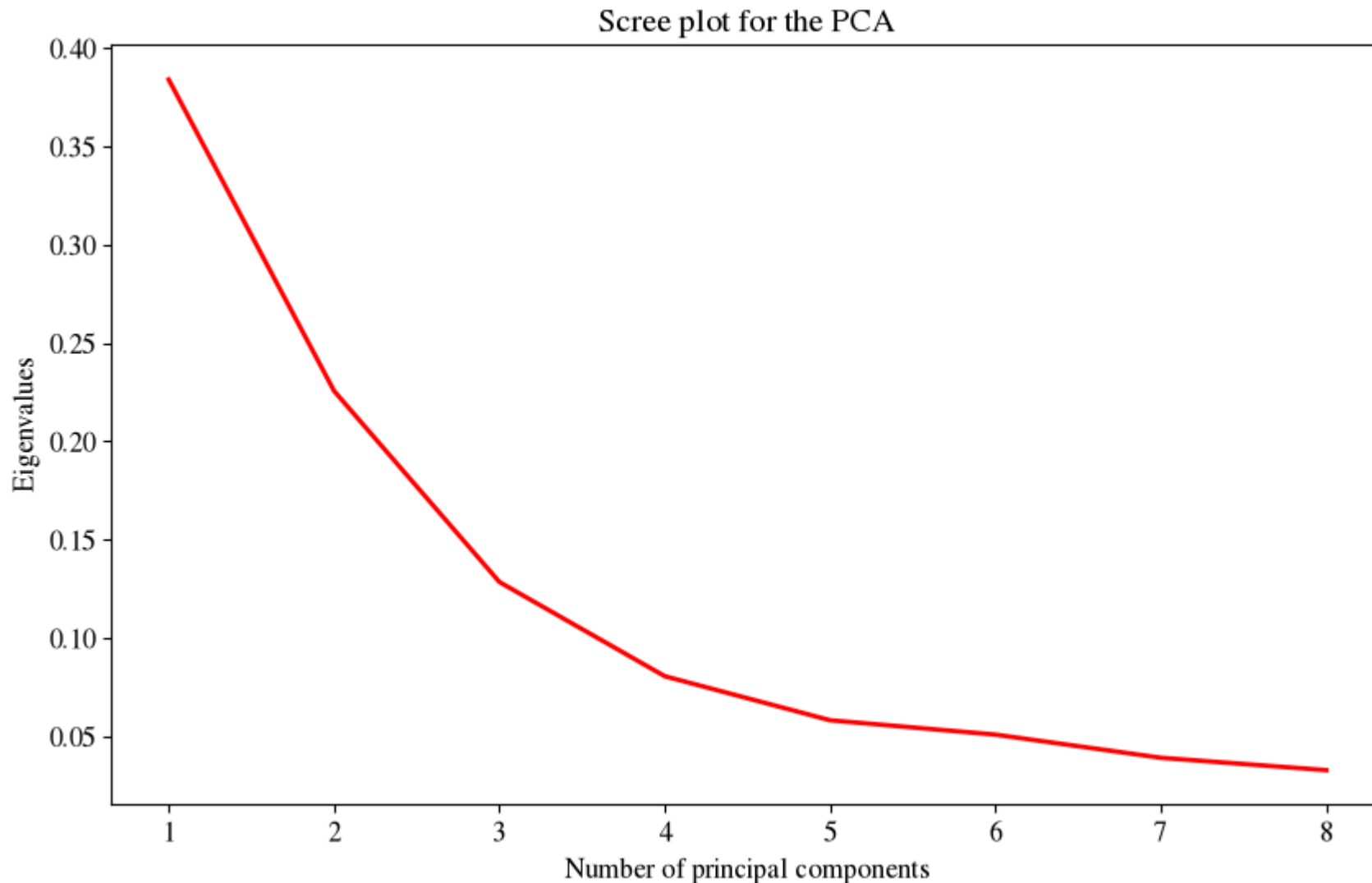
```
400 4096
(400, 400)
```

```
In [83]: eig = eigenvalues/np.sum(eigenvalues)
    var_exp = eig.cumsum()*100.

    plt.plot(np.arange(1,len(eig)+1), eig, color='r')
    plt.title('Scree plot for the PCA')
```

```
plt.xlabel('Number of principal components')
plt.ylabel('Eigenvalues')
plt.show()

print ('The 1st Principal Component explains {:.03.1f} % of the variance\n'.format(var_exp[0]))
print ('The 1st and 2nd Principal Components explain {:.03.1f} % of the variance\n'.format(var_exp[1]))
print ('The 1st, 2nd and 3rd Principal Components explain {:.03.1f} % of the variance\n'.format(var_exp[2]))
print ('The first four Principal Components explain {:.03.1f} % of the variance\n'.format(var_exp[3]))
print ('The first five Principal Components explain {:.03.1f} % of the variance\n'.format(var_exp[4]))
print ('The first six Principal Components explain {:.03.1f} % of the variance\n'.format(var_exp[5]))
```



The 1st Principal Component explains 38.4 % of the variance

The 1st and 2nd Principal Components explain 61.0 % of the variance

The 1st, 2nd and 3rd Principal Components explain 73.8 % of the variance

The first four Principal Components explain 81.9 % of the variance

The first five Principal Components explain 87.7 % of the variance

The first six Principal Components explain 92.8 % of the variance

```
In [84]: plt.figure(figsize=(4,4))
plt.axis('off')
plt.gray()
_ = plt.imshow(mean_I.reshape(m,n))
```



```
In [85]: def pca_reconstruction(image, numEvs):
weights = np.dot( I[image].reshape(1,-1) - mean_I, eigenvectors[:,0: numEvs ])
#reconstruct using principal components
recon = mean_I + np.dot( weights, eigenvectors[:,0: numEvs ].T)
plt.figure(figsize=(4,4))
plt.axis('off')
plt.imshow(recon.reshape(m,n))
plt.show()
```

```
pca_reconstruction(30, 8)
```



```
In [86]: num_components = 400  
          eigenvalues, eigenvectors, mean_I = pca_calculation(I, num_components)  
          pca_reconstruction(30, num_components)
```

```
400 4096  
(400, 400)
```



Alternative: Using Scikit decomposition PCA - to show that that same thing can be calculated simply using the package

```
In [87]: X = I
print(X.shape[0:2])
n_components = 400 # What happens if we don't use 400?

pca = sklearn.decomposition.PCA(n_components)
# X is the matrix transposed (n samples on the rows, m features on the columns)
pca.fit(X)

# compared to eigenvalues, eigenvectors, mean_I = pca_calculation(I, num_components)
X_proj = pca.transform(X)
mean_I = pca.mean_
eigenvectors = pca.components_.T

plt.figure(figsize=(4,4))
plt.axis('off')
_ = plt.imshow(pca.mean_.reshape(m,n))
```

(400, 4096)



```
In [88]: pca_reconstruction(30,4)
```



```
In [89]: pca_reconstruction(30,8)
```



```
In [90]: pca_reconstruction(30,400) # What happened here?
```



```
In [91]: # What do the components themselves look like?
for i in [0,1,2,3,4,5]:
    plt.figure(figsize=(4,4))
    plt.axis('off')
```

```
plt.imshow(pca.components_[i,:].reshape(m,n))  
plt.show()
```

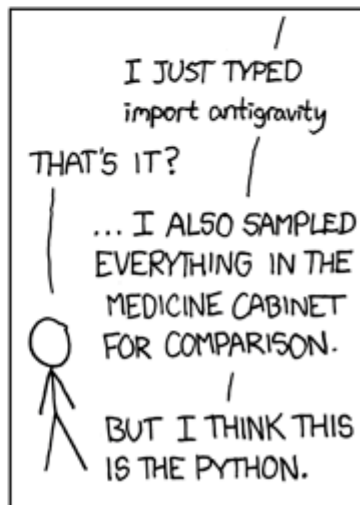
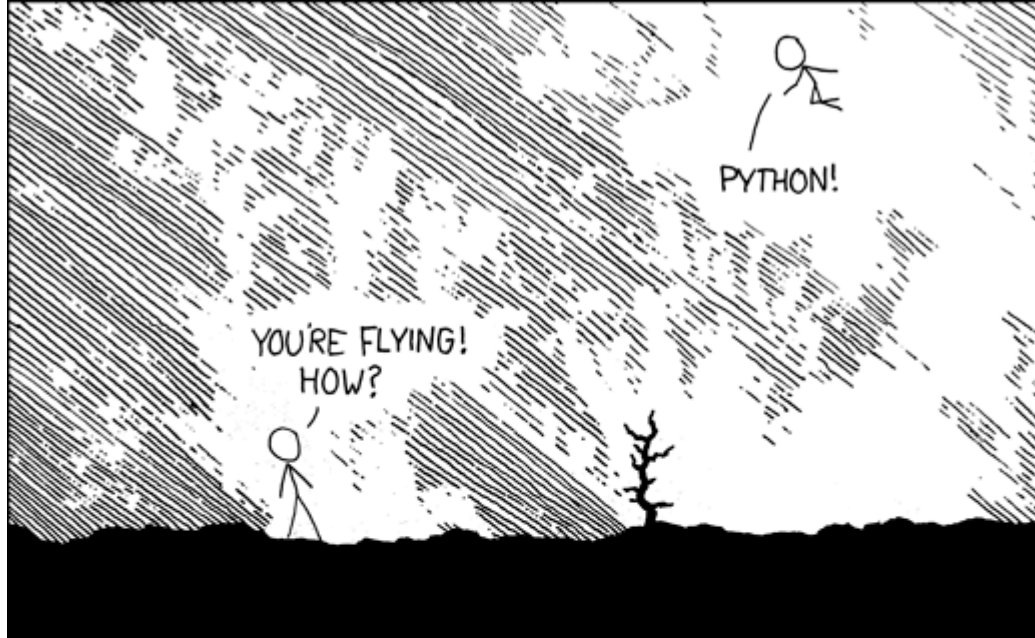






[back to top](#)

Overview of Python, Pandas and Matplotlib



Some rights reserved -- xkcd.com

In [92]: `# Python Libraries Test`

```
test = True
if test:
    print("Hello World!")
```

Hello World!

```
In [93]: import sys
print(sys.version)
```

3.13.0 | packaged by conda-forge | (main, Nov 27 2024, 19:18:26) [Clang 18.1.8]

```
In [94]: #IPython is what you are using now to run the notebook
import IPython
print("IPython version:      %6.6s " % IPython.__version__)

# Numpy is a library for working with Arrays
import numpy
print("Numpy version:        %6.6s " % numpy.__version__)

# SciPy implements many different numerical algorithms
import scipy
print("SciPy version:        %6.6s " % scipy.__version__)

# Pandas makes working with data tables easier
import pandas
print("Pandas version:       %6.6s " % pandas.__version__)

# Module for plotting
import matplotlib
print("Matplotlib version:   %6.6s " % matplotlib.__version__)

# SciKit Learn implements several Machine Learning algorithms
import sklearn
print("Scikit-Learn version: %6.6s " % sklearn.__version__)
```

```
IPython version:      9.17.1
Numpy version:        2.5.2
SciPy version:        1.18.0
Pandas version:       3.0.5
Matplotlib version:   3.11.1
Scikit-Learn version: 1.9.0
```

Missing packages? - use conda /pip

```
conda install numpy
conda install matplotlib
```

or

```
pip install numpy
pip install matplotlib
```

Python Lists and Dictionaries

One of the first things that people new to Python discover, even if they have experience with Object Oriented Programming languages or scripting languages like Javascript, is that Python has a certain way of doing things - which place it kind of in-between both of the latter worlds. Two of the key elements are lists and dictionaries. Understanding both will help you to write efficient "Pythonic" code. Lists are used for list comprehensions, the contents of open files, dictionaries, and as part of a general notion of something called an iterator.

Let's introduce the notion of a comprehension. It's a way of constructing a list

```
In [95]: # Make a list from a sequence of integers
alist=[1,2,3,4,5]
print (alist)

# Perform an operation - squaring - on each element of the list
alist_squared=[i*i for i in alist]
print (alist_squared)
```

```
[1, 2, 3, 4, 5]
[1, 4, 9, 16, 25]
```

That's neat. But what else can we do with lists?

Enumerate and zip

There are useful functions like enumerate and zip which allow you to count and to combine lists

```
In [96]: enumerate(alist_squared)
```

```
Out[96]: <enumerate at 0x306d5ac50>
```

...strange!? But wait:

```
In [97]: [(idx+1, value_squared) for idx, value_squared in enumerate(alist_squared)]
```

```
Out[97]: [(1, 1), (2, 4), (3, 9), (4, 16), (5, 25)]
```

Combine two lists to make tuples

Zip is a pythonic way of combining lists to create a new list of tuples

```
In [98]: [(elem) for elem in zip(alist, alist_squared)] # zip to stitch elements together
```

```
Out[98]: [(1, 1), (2, 4), (3, 9), (4, 16), (5, 25)]
```

```
In [99]: print(zip(alist, alist_squared))
```

```
<zip object at 0x3069f8380>
```

```
In [100]: print(list(zip(alist, alist_squared)))
```

```
[(1, 1), (2, 4), (3, 9), (4, 16), (5, 25)]
```

What is the '*' star symbol for?

To unpack values from a list (from the documentation)

```
In [101]: list(range(3, 6))    # normal call with separate arguments
args = [3, 6]
list(range(*args))           # call with arguments unpacked from a list (When you don't know # of parameters to your code)
```

```
Out[101]: [3, 4, 5]
```

Conditional statements

We will use Python with conditional statements, but watch the tab-spacing and the use of parentheses

```
In [102]: newlist = [1,2,3,4,5]

if 9 not in newlist:
    print("9 is not in the list")
```

```
9 is not in the list
```

```
In [103]: if 9 in newlist == False:    # notice the capital F in False
    print("9 is not in the list")    #Something weird happening here, you can analyze AST (Abstract syntax tree) in python
#import ast
#ast.dump(ast.parse("9 in newlist == False"))
#In short do not do this - use parenthesis like in following cell.
```

```
In [104]: if (9 in newlist) == False:
    print("9 is not in the list")
```

```
9 is not in the list
```

Download a file from an URL

We can download a file from the web, using the requests packages

Let's see with the [Pokemon Go Dataset](#)

```
In [105... import requests #pip install requests #if needed

url = 'https://raw.githubusercontent.com/ChihChengLiang/pokemongor/master/data-raw/pokemons.csv'

r = requests.get(url) #We are not going into details of scrpaing but please be mindful of scraping etiquette
open('pokemons.csv', 'wb').write(r.content)
```

Out[105... 40910

File contents

The contents of an open file is also a list, so a .csv file is just a list of lines

```
In [106... # each line of the file read is an element of the list
linelengths=[len(line.strip()) for line in open("pokemons.csv", encoding="utf-8")]
print(linelengths[:50])
```

[446, 273, 283, 279, 277, 281, 280, 261, 274, 271, 234, 250, 264, 239, 249, 269, 267, 279, 276, 261, 262, 265, 263, 256, 252, 275, 273, 261, 271, 272, 270, 268, 267, 278, 274, 279, 270, 270, 272, 283, 282, 265, 267, 261, 276, 269, 265, 268, 272, 268]

Dictionaries

```
In [107... adict={'one':1, 'two': 2, 'three': 3, 'four': 4} # curly brackets!!

print("Print the keys:")
print([i for i in adict]) # gives the key of the dict
#OR
print(list(adict.keys()))

print("\nPrint the tuples:")
print([(k,v) for k,v in adict.items()]) # gives each tuple of the dict - note use iteritems() for Python 2.7

print("\nPrint the values:")
print(adict.values()) # gives the values
```

Print the keys:

```
['one', 'two', 'three', 'four']
```

```
['one', 'two', 'three', 'four']
```

Print the tuples:

```
[('one', 1), ('two', 2), ('three', 3), ('four', 4)]
```

Print the values:

```
dict_values([1, 2, 3, 4])
```

```
In [108... #You may want to increment the value of one element of the dict.  
#Just use its key to reference it:
```

```
print (adict)  
for i in range(3):  
    adict['one'] += 1  
    print (adict['one'])  
  
print (adict)
```

```
{'one': 1, 'two': 2, 'three': 3, 'four': 4}
```

```
2
```

```
3
```

```
4
```

```
{'one': 4, 'two': 2, 'three': 3, 'four': 4}
```

Populating lists

`range([start], stop[, step])` is a Python way of creating a sequence, can be used to populate a list.

*Note: `range()` in Python 3.X acts like `xrange()` in Python 2.X and the original `range()` from Python 2.X has been deprecated *

Here we start with an empty list, and then append the elements

```
In [109... mylist=[] # empty list  
for i in range(10): # a for loop  
    mylist.append(i) # append each additional element to the end of mylist  
  
print (mylist)
```

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

You may also want to simply concatenate lists, which is simple in Python.

You can also perform more complicated manipulations, look up [itertools](#)

```
In [110... mylist=[] # empty list
for i in range(10): # a for loop
    mylist = mylist + [i] # concatenate additional elements as a list

print (mylist)
```

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Dictionary comprehensions

Like lists, you can construct dictionaries using dictionary comprehensions (from Python 2.7 onwards). Here we use the elements of each tuple in the zipped list to create the keys and values of the dict:

```
In [111... {v:k for (k,v) in zip(alist, alist_squared)}
```

```
Out[111... {1: 1, 4: 2, 9: 3, 16: 4, 25: 5}
```

A dict can also contain values that are lists themselves

```
In [112... adict={'one':[1,2,3,4], 'two': [5,6,7,8], 'three': [2,3,4], 'four': [0,2,4,6]}
print(adict.keys(), '\n')
for i, k in zip(range(len(adict.keys())), adict.keys()):
    print (i, k)
print()
print(adict.keys() )# print out all the keys
print()
print (adict['one']) # print out the list stored in 'one'
```

```
dict_keys(['one', 'two', 'three', 'four'])
```

```
0 one
1 two
2 three
3 four
```

```
dict_keys(['one', 'two', 'three', 'four'])
```

```
[1, 2, 3, 4]
```

Lambda functions

Lambda functions are anonymous functions, typically small and throw-away functions, that have become popular in other scripting languages and are available in Python too. They can be quite useful in some Python APIs for other software (e.g. Spark, Tensorflow), in particular when used with

`filter()`, `map()` and `reduce()` .

Syntax:

```
[returned_variable] = lambda argument_list: expression
```

```
In [113... add_func = lambda x, y : x + y
add_func(3,4)
```

Out[113... 7

```
In [114... #This is equivalent to above lambda function
def add_func(x,y):
    return x+y

add_func(3,4)
```

Out[114... 7

```
In [115... mylist2 = [1,2,3,4,5,6,7,8,9]
mapped = map(lambda x:x+2, mylist2)
[i for i in mapped]
```

Out[115... [3, 4, 5, 6, 7, 8, 9, 10, 11]

Copying - shallow or deep copy

```
In [116... a = [[1, 2, 3], [4, 5, 6]]
b = a # list(a) also returns a shallow copy
print (a, '\n', b)
```

```
[[1, 2, 3], [4, 5, 6]]
[[1, 2, 3], [4, 5, 6]]
```

```
In [117... a[1][2] = 10
print (a, '\n', b)
```

```
[[1, 2, 3], [4, 5, 10]]
[[1, 2, 3], [4, 5, 10]]
```

When a deep copy is needed, use `deepcopy()` instead

<https://docs.python.org/3/library/copy.html>

```
In [118... from copy import copy, deepcopy
a = [[1, 2, 3], [4, 5, 6]]
b = copy(a) # copy() returns a shallow copy
```

```
c = deepcopy(a) # deepcopy() returns a deep copy
a[1][2] = 10
print (a, b, c)
```

```
[[1, 2, 3], [4, 5, 10]] [[1, 2, 3], [4, 5, 10]] [[1, 2, 3], [4, 5, 6]]
```

Numpy

NumPy is the fundamental package for scientific computing with Python. It provides a high-performance multidimensional array object, and tools for working with these arrays. Numpy adds support for large, multi-dimensional arrays and matrices, along with a large library of high-level mathematical functions to operate on these arrays.

Here is a [link](#) to a good Numpy tutorial.

```
In [119... import numpy as np
```

Creating arrays

```
In [120... a = np.array([3, 5, 7, 9]) # 1D array
b = np.array([[4,5,6,2], [7,3,4,1]]) # 2D array: 2 rows X 4 columns
print(a, '\n')
print(b)
```

```
[3 5 7 9]
```

```
[[4 5 6 2]
 [7 3 4 1]]
```

```
In [121... print(a.shape)
print(b.shape)
```

```
(4,)
(2, 4)
```

Creating arrays with numpy arange()

[numpy.arange](#) Return evenly spaced values within a given interval.

```
In [122... d = np.arange(6) #numpy.arange([start, ]stop, [step, ])
print(d, '\n')
d2 = np.arange(-4, 4, 0.5)
print(d2, '\n')
```

```
d3 = np.arange(15, 8, -1)
print(d3)
```

```
[0 1 2 3 4 5]
```

```
[-4.  -3.5 -3.  -2.5 -2.  -1.5 -1.  -0.5  0.   0.5  1.   1.5  2.   2.5
 3.   3.5]
```

```
[15 14 13 12 11 10  9]
```

Creating arrays with numpy linspace() and logspace()

[numpy.logspace](#)

[numpy.linspace](#) Evenly spaced numbers with careful handling of endpoints.

<http://stackoverflow.com/questions/5779270/linspace-vs-range>

```
In [123... a = np.linspace(-2, 2,10) #numpy.linspace(start, stop, num)
print(a, '\n')
b = np.logspace(0, 3,5)      #numpy.logspace(start, stop, num, base=10.0)
print(b, '\n')
```

```
[-2.          -1.55555556 -1.11111111 -0.66666667 -0.22222222  0.22222222
 0.66666667  1.11111111  1.55555556  2.          ]
```

```
[ 1.          5.62341325 31.6227766 177.827941 1000.          ]
```

1-D Arrays

Watch out for 1-D arrays - Numpy creates 1-D arrays, with the second dimension empty

```
In [124... a.shape
```

```
Out[124... (10,)
```

Here are some ways to reshape the 1-D array

```
In [125... print(np.expand_dims(a, axis=1).shape)
print(a[:,np.newaxis].shape)
print(a[:,None].shape)
print(a.reshape(10,1).shape)
```

```
(10, 1)
(10, 1)
(10, 1)
(10, 1)
```

ndarray indexing and ranges m:n - array indices begin at 0; ranges refer from m to n-1

In [126...

```
a = np.array([7, 4,5,2,8,1,9,0,2])
print("a = ", a)
print("a[2:5] = ", a[2:5])
print('')
print("a[5:] = " ,a[5:])
print('')
print("a[:4] = " , a[:4])
print('')
print(a[::-1]) # reverse the array
print()

b = np.array([[4,5,6,2], [7,3,4,1]]) # 2D array: 2 rows X 4 columns
print("b = \n", b, '\n')
print("b[1,3] = ", b[1,3])
print('')
print("b[:,2] = " , b[:,2]) #All rows , 2nd column
```

```
a = [7 4 5 2 8 1 9 0 2]
a[2:5] = [5 2 8]
```

```
a[5:] = [1 9 0 2]
```

```
a[:4] = [7 4 5 2]
```

```
[2 0 9 1 8 2 5 4 7]
```

```
b =
[[4 5 6 2]
 [7 3 4 1]]
```

```
b[1,3] = 1
```

```
b[:,2] = [6 4]
```

Reshape() and Transposing Arrays

In [127...

```
ar1 = np.arange(15)
print("ar1 -> ", ar1, '\n')
```

```
ar2 = np.arange(15).reshape(3,5)
print("ar2 (After reshape(3,5) \n",ar2, '\n')

print("Transpose of ar2 -> \n" , ar2.T) #returns the transpose of ar2
```

```
ar1 -> [ 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14]
```

```
ar2 (After reshape(3,5)
```

```
[[ 0  1  2  3  4]
 [ 5  6  7  8  9]
 [10 11 12 13 14]]
```

```
Transpose of ar2 ->
```

```
[[ 0  5 10]
 [ 1  6 11]
 [ 2  7 12]
 [ 3  8 13]
 [ 4  9 14]]
```

Numpy functions for creating arrays of zero's, one's and empty array

In [128...

```
a = np.zeros(10) #creates arrays of 10 zeros
print(a,'\n')
b = np.zeros((4,3)) #Pass tuples to create matrix
print(b,'\n')
c = np.ones((3,4), dtype = np.int64)
print(c, '\n')
print(6*c, '\n')
d = np.eye(4, dtype = np.float64) # Identity matrix
print(d,'\n')

a = np.empty((5,3), dtype = np.float64)
print(a)
```

```
...
```

Notes

empty, unlike zeros, does not set the array values to zero, and may therefore be marginally faster. On the other hand, it requires the user to manually set all the values in the array, and should be used with caution.

```
...
```

```
[0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
```

```
[[0. 0. 0.]  
 [0. 0. 0.]  
 [0. 0. 0.]  
 [0. 0. 0.]]
```

```
[[1 1 1 1]  
 [1 1 1 1]  
 [1 1 1 1]]
```

```
[[6 6 6 6]  
 [6 6 6 6]  
 [6 6 6 6]]
```

```
[[1. 0. 0. 0.]  
 [0. 1. 0. 0.]  
 [0. 0. 1. 0.]  
 [0. 0. 0. 1.]]
```

```
[[ 0.  0. nan]  
 [ 0. nan  0.]  
 [ 0. nan  0.]  
 [nan  0.  0.]  
 [ 0.  0.  0.]]
```

Out[128... '\nNotes\nempty, unlike zeros, does not set the array values to zero, and may therefore be marginally \nfaster. On the other h
and, it requires the user to manually set all the values in the array, \nand should be used with caution.\n'

Casting to int, round() and floor()

```
In [129... arr = np.array([3.7, -1.5, 2.8, -4.6], dtype=np.float64)  
print(arr, '\n')  
arr2 = arr.astype(np.int32)  
print(arr2, '\n')  
arr3 = np.round(arr)  
print(arr3, '\n')  
arr4 = np.floor(arr)  
print(arr4, '\n')
```

```
[ 3.7 -1.5  2.8 -4.6]
```

```
[ 3 -1  2 -4]
```

```
[ 4. -2.  3. -5.]
```

```
[ 3. -2.  2. -5.]
```

String to float casting

```
In [130... arr = np.array(['3.7', '-1.5', '2.8', '-4.6'], dtype = np.bytes_)
arr2 = arr.astype(float) # python recognizes it as arr.astype(np.float64)
arr2
```

```
Out[130... array([ 3.7, -1.5,  2.8, -4.6])
```

Maxtrix multiplication and Vectorization -

Multidimensional arrays allow us operations on data they contain with out the need to write loops such as for loops - a process known as vectorization

- replacing explicit loops with array expression
- vectorized array operations are faster

1D numpy array - elementwise operations

```
In [131... a1 = np.array([2.5, 5., 3.2])
print("a1 = " , a1, '\n')
print("np.ones(3) = ", np.ones(3), '\n')
a2 = 2*np.ones(3)
print("a2 = 2*np.ones(3) = ", a2, '\n')

a3 = a1*a2 # element by element multiplication
print("a3 = a1*a2 = " , a3, '\n')

a4 = 1/a1 # element-by-element division
print("a4 = 1/a1" , a4)
```

```
a1 = [2.5 5. 3.2]
```

```
np.ones(3) = [1. 1. 1.]
```

```
a2 = 2*np.ones(3) = [2. 2. 2.]
```

```
a3 = a1*a2 = [ 5. 10. 6.4]
```

```
a4 = 1/a1 [0.4 0.2 0.3125]
```

2D array - elementwise operations

```
In [132... b1 = np.array([[1,2,3], [4,5,6], [6,7,8]])
print("b1 = \n", b1, "\n")

b2 = 2*b1
print("b2 = 2*b1 = \n",b2, "\n")

b3 = b1*b2 # element by element multiplication
print("b3 = b1*b2 = \n", b3 , "\n")

b4 = 1/b1 # element-by-element division
print("b4 = 1/b1 = \n" ,b4,"\n")

b5 = b1 ** 2 #elementwise squaring
print("b5 = b1 ** 2 = \n" ,b5 ,"\n")
```

```
b1 =  
[[1 2 3]  
 [4 5 6]  
 [6 7 8]]
```

```
b2 = 2*b1 =  
[[ 2  4  6]  
 [ 8 10 12]  
 [12 14 16]]
```

```
b3 = b1*b2 =  
[[ 2  8 18]  
 [32 50 72]  
 [72 98 128]]
```

```
b4 = 1/b1 =  
[[1.         0.5         0.33333333]  
 [0.25        0.2         0.16666667]  
 [0.16666667 0.14285714 0.125        ]]
```

```
b5 = b1 ** 2 =  
[[ 1  4  9]  
 [16 25 36]  
 [36 49 64]]
```

You can also create a matrix with `np.mat`, by simply entering a semi-colon separated string (i.e. list) of numbers.

```
In [133... b6 = np.asmatrix('1,2,3;4,5,6;6,7,8')  
print("b6 = \n", b6, "\n")
```

```
b6 =  
[[1 2 3]  
 [4 5 6]  
 [6 7 8]]
```

Consider the following

a1 is
[2.5 5. 3.2]

b1 is
[[1 2 3]

```
[4 5 6]
[6 7 8]]
```

```
In [134... print("multiply array and matrix \n", a1*b1, "\n")

print("add a new axis to an array \n", a1[:,np.newaxis]*b1, "\n")
print("shape of a1 with new axis \n", a1[:,np.newaxis].shape)
```

multiply array and matrix

```
[[ 2.5 10.   9.6]
 [10.  25.  19.2]
 [15.  35.  25.6]]
```

add a new axis to an array

```
[[ 2.5  5.   7.5]
 [20.  25.  30. ]
 [19.2 22.4 25.6]]
```

shape of a1 with new axis

```
(3, 1)
```

Random number generation and matrix multiplication

```
In [135... np.random.seed(123)    #Seed the generator (Helps debugging code with random numbers)
arr = np.random.randn(6, 3)
print(arr)
```

```
[[-1.0856306   0.99734545  0.2829785 ]
 [-1.50629471 -0.57860025  1.65143654]
 [-2.42667924 -0.42891263  1.26593626]
 [-0.8667404   -0.67888615 -0.09470897]
 [ 1.49138963 -0.638902   -0.44398196]
 [-0.43435128  2.20593008  2.18678609]]
```

`np.dot` Dot product of two arrays.

For 2-D arrays it is equivalent to matrix multiplication, and for 1-D arrays to inner product of vectors

```
In [136... a1 = np.array([2.5, 5., 3.2])
print("a1 = " , a1, '\n')
print("np.ones(3) = ", np.ones(3), '\n')
a2 = 2*np.ones(3)
print("a2 = 2*np.ones(3) = ", a2, '\n')
print("np.dot(a1,a2) = ", np.dot(a1,a2), "\n")
```

```
a1 = [2.5 5. 3.2]
```

```
np.ones(3) = [1. 1. 1.]
```

```
a2 = 2*np.ones(3) = [2. 2. 2.]
```

```
np.dot(a1,a2) = 21.4
```

$$np.dot \text{ with } 2d \text{ array} \Rightarrow a = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} b = \begin{bmatrix} 4 & 1 \\ 2 & 2 \end{bmatrix} \quad np.dot(a,b) = \begin{bmatrix} 6 & 3 \\ 6 & 3 \end{bmatrix}$$

```
In [137... a = np.array([[1, 1], [1, 1]])
print("a = \n",a,"\n")
b = np.array([[4, 1], [2, 2]])
print("b = \n",b,"\n")
print("np.dot(a, b) = \n", np.dot(a,b),"\n")
```

```
a =
[[1 1]
 [1 1]]
```

```
b =
[[4 1]
 [2 2]]
```

```
np.dot(a, b) =
[[6 3]
 [6 3]]
```

Matrix inverses

The Numpy methods in `linalg` can be used to calculate the matrix inverse. Here for example, for the previous matrix `b`.

```
In [138... m_inv = np.linalg.inv(b)
print(m_inv)
print()
print (np.allclose(np.dot(b, m_inv), np.eye(2)))
```

```
[[ 0.33333333 -0.16666667]
 [-0.33333333  0.66666667]]
```

```
True
```

Note: You can also access the inverse using the 'I' method in `numpy.linalg` on matrix objects

```
In [139... b_mat = np.asmatrix(b)
print(b_mat.I)

[[ 0.33333333 -0.16666667]
 [-0.33333333  0.66666667]]
```

Timing your algorithms

You can use the `time` and `timeit` modules to calculate the elapsed time of your algorithms. The best example is to show the speed increase between a python list comprehension (i.e. for loop) and a vectorized operation in Numpy.

This example comes from this short online [book on Numpy](#)

```
In [140... # make a simple Python function with addition in a list comprehension
def add_python(Z1,Z2):
    return [z1+z2 for (z1,z2) in zip(Z1,Z2)]

# make a simple Numpy function using the vectorized add method
def add_numpy(Z1,Z2):
    return np.add(Z1,Z2)
```

```
In [141... import time

Z1 = np.random.sample(1000)*100000
Z2 = np.random.sample(1000)*100000

starttime = time.time()
add_python(Z1, Z2)
endtime = time.time()
print("With the Python list comprehension it took {:.3f} seconds\n".format(endtime-starttime))

starttime = time.time()
add_numpy(Z1, Z2)
endtime = time.time()
print("With the Numpy add method it took {:.3f} seconds\n".format(endtime-starttime))
```

With the Python list comprehension it took 0.000712872 seconds

With the Numpy add method it took 0.000164270 seconds

With the `timeit` magic in Jupyter, you pass the function to `timeit` as an argument - this is useful if you have several parts to time and you don't want to do the math to find the deltas.

You can find details about `timeit` module [here](#), but basically, the `%timeit` magic allows you the power of the module, but with a simplified call in Jupyter.

```
In [142... %timeit (add_python(Z1, Z2))  
  
%timeit (add_numpy(Z1, Z2))
```

```
57.2 µs ± 759 ns per loop (mean ± std. dev. of 7 runs, 10,000 loops each)  
398 ns ± 4.15 ns per loop (mean ± std. dev. of 7 runs, 1,000,000 loops each)
```

pandas

Pandas is the primary data analysis package for Python, created by Wes McKinney in order to do econometrics and financial time series in python, using the dataframe concept. The online documentation is not the greatest, but in contrast, most people who use pandas have at one time read Wes' wonderful book, [Python for Data Analysis](#). Check on HOLLIS!

```
In [143... import pandas as pd
```

Pokemon Go Dataset : [Source](#)

[pandas.read_csv](#) - Read CSV (comma-separated) file into DataFrame

Few commonly used parameters with `read_csv`:

- `sep` : default `' , '`
- `nrows` : `int` , default `None` Number of rows of file to read. Useful for reading pieces of large files
- `header` , `index_col` , `usecols` , `dtype`

Warning: `sep` , `delimiter` , `delim_whitespace` , `header`

Most of the difficulties that you will encounter with `read_csv` trying to download/load a file will typically be related to getting the first few options right - this pertains to whether there is a header in the CSV or not (`header=None`) and the type of character used to delimit the data, e.g. comma, semi-colon, or frequently, a tab (`'\t'`).

For example, some of the `.data` files in the very good [UCI machine learning repository](#) appear to be tab-separated, but often need loading using the `delim_whitespace=True` option.

```
In [144... #Read a csv file
pokemonData = pd.read_csv("pokemons.csv")
```

```
In [145... #Number of Rows and Columns
pokemonData.shape
```

Out[145... (151, 31)

```
In [146... pokemonData.head(n=7) #Top 7 rows (default is 5)
```

Out[146...

	PkMn	Identifier	EvolvesFrom	EvolvesTo	EvoChainID	EvoStage	EvolutionPips	BaseStamina	BaseAttack	BaseDefense	...	QuickMoves
0	1	Bulbasaur	0	Ivysaur	1	1	NORMAL	90	126	126	...	Vine Whip, Tackle
1	2	Ivysaur	Bulbasaur	Venusaur	1	2	NORMAL	120	156	158	...	Razor Leaf, Vine Whip
2	3	Venusaur	Ivysaur	0	1	3	NORMAL	160	198	200	...	Razor Leaf, Vine Whip
3	4	Charmander	0	Charmeleon	2	1	NORMAL	78	128	108	...	Ember, Scratch
4	5	Charmeleon	Charmander	Charizard	2	2	NORMAL	116	160	140	...	Ember, Scratch
5	6	Charizard	Charmeleon	0	2	3	NORMAL	156	212	182	...	Ember, Wing Attack
6	7	Squirtle	0	Wartortle	3	1	NORMAL	88	112	142	...	Bubble, Tackle

7 rows x 31 columns

```
In [147... #List of columns
pokemonData.columns
```

```
Out[147... Index(['Pkmn', 'Identifier', 'EvolvesFrom', 'EvolvesTo', 'EvoChainID',
      'EvoStage', 'EvolutionPips', 'BaseStamina', 'BaseAttack', 'BaseDefense',
      'Type1', 'Type2', 'BaseCaptureRate', 'BaseFleeRate', 'CollisionRadiusM',
      'CollisionHeightM', 'CollisionHeadRadiusM', 'MovementType',
      'MovementTimerS', 'JumpTimeS', 'AttackTimerS', 'QuickMoves',
      'CinematicMoves', 'PokemonClass', 'PokedexHeightM', 'PokedexWeightKg',
      'HeightStdDev', 'WeightStdDev', 'CandyFamily', 'CandyToEvolve',
      'AnimTime'],
      dtype='str')
```

```
In [148... #To find unique values in a data frame column
print("EvolutionPips = ",pd.unique(pokemonData.EvolutionPips))
```

```
print("Type1 = ", pd.unique(pokemonData.Type1))
```

```
EvolutionPips = <StringArray>
['NORMAL', 'LEGENDARY', 'MYTHIC']
Length: 3, dtype: str
Type1 = <StringArray>
[  'GRASS',      'FIRE',      'WATER',      'BUG',      'NORMAL',      'POISON',
  'ELECTRIC',    'GROUND',    'FAIRY',    'FIGHTING', 'PSYCHIC',    'ROCK',
   'GHOST',      'ICE',      'DRAGON']
Length: 15, dtype: str
```

```
In [149... #Frequency counts for Type1
pokemonData.Type1.value_counts()
```

```
Out[149... Type1
WATER      28
NORMAL     22
POISON     14
GRASS      12
FIRE       12
BUG        12
ELECTRIC    9
ROCK        9
GROUND      8
PSYCHIC     8
FIGHTING    7
GHOST       3
DRAGON      3
FAIRY       2
ICE         2
Name: count, dtype: int64
```

In [150...

#List all Pokemons where Type1 = ICE
pokemonData[pokemonData.Type1=="ICE"]

Out[150...

	PkMn	Identifier	EvolvesFrom	EvolvesTo	EvoChainID	EvoStage	EvolutionPips	BaseStamina	BaseAttack	BaseDefense	...	QuickMoves
123	124	Jynx	0	0	59	1	NORMAL	130	172	134	...	Frost Breath, Pound
143	144	Articuno	0	0	73	1	LEGENDARY	180	198	242	...	Frost Breath

2 rows × 31 columns



In [151...

#Top10 BaseCapture rate
pokemonData.sort_values('CandyToEvolve',ascending=False)[['Identifier','CandyToEvolve']][:10]

Out[151...

	Identifier	CandyToEvolve
128	Magikarp	400
92	Haunter	100
32	Nidorino	100
29	Nidorina	100
60	Poliwhirl	100
63	Kadabra	100
66	Machoke	100
69	Weepinbell	100
74	Graveler	100
1	Ivysaur	100

In [152...

#Average PokedexWeightKg by Type1
pokeGrpWeight = pokemonData.groupby(['Type1']).agg({'PokedexWeightKg':'mean'})
pokeGrpWeight #Here index is set to Type1

Out [152...

PokedexWeightKg	
Type1	
BUG	22.991667
DRAGON	76.600000
ELECTRIC	31.788889
FAIRY	23.750000
FIGHTING	54.285714
FIRE	48.025000
GHOST	13.566667
GRASS	27.991667
GROUND	45.262500
ICE	48.000000
NORMAL	50.086364
POISON	27.314286
PSYCHIC	51.562500
ROCK	87.611111
WATER	57.967857

In [153...

```
pokeGrpCount = pokemonData.groupby(['Type1']).agg({'Identifier': 'count'})
pokeGrpCount #Here index is set to Type1
```

Out [153...

Identifier	
Type1	
BUG	12
DRAGON	3
ELECTRIC	9
FAIRY	2
FIGHTING	7
FIRE	12
GHOST	3
GRASS	12
GROUND	8
ICE	2
NORMAL	22
POISON	14
PSYCHIC	8
ROCK	9
WATER	28

In [154...

```
#Many times we need to reset the index, e.g: merging two dataframes
pokeGrpWeight.reset_index(inplace=True)
pokeGrpCount.reset_index(inplace=True)

pd.merge(pokeGrpWeight, pokeGrpCount)
```

Out [154...

	Type1	PokedexWeightKg	Identifier
0	BUG	22.991667	12
1	DRAGON	76.600000	3
2	ELECTRIC	31.788889	9
3	FAIRY	23.750000	2
4	FIGHTING	54.285714	7
5	FIRE	48.025000	12
6	GHOST	13.566667	3
7	GRASS	27.991667	12
8	GROUND	45.262500	8
9	ICE	48.000000	2
10	NORMAL	50.086364	22
11	POISON	27.314286	14
12	PSYCHIC	51.562500	8
13	ROCK	87.611111	9
14	WATER	57.967857	28

In [155...

```
#Find all pokemons where 'CinematicMoves' contains 'Fire Blast'
pokemonData[pokemonData.CinematicMoves.str.contains('Fire Blast')][['Identifier','CinematicMoves','QuickMoves']]
```

Out [155...

	Identifier	CinematicMoves	QuickMoves
5	Charizard	Fire Blast, Dragon Claw, Flamethrower	Ember, Wing Attack
37	Ninetales	Heat Wave, Flamethrower, Fire Blast	Feint Attack, Ember
58	Arcanine	Fire Blast, Flamethrower, Bulldoze	Fire Fang, Bite
76	Ponyta	Flame Charge, Flame Wheel, Fire Blast	Tackle, Ember
77	Rapidash	Fire Blast, Drill Run, Heat Wave	Low Kick, Ember
125	Magmar	Fire Blast, Fire Punch, Flamethrower	Ember, Karate Chop
135	Flareon	Fire Blast, Flamethrower, Heat Wave	Ember
145	Moltres	Fire Blast, Heat Wave, Flamethrower	Ember
150	Mew	Hurricane, Earthquake, Psychic, Dragon Pulse, ...	Pound

Matplotlib

Matplotlib is very extensive library with many customization possibilities. Check out the [documentation](#) for a complete overview. There is also a very good tutorial in a Jupyter Notebook [here](#) which goes into more detail than we will in this Section.

You don't need to use Seaborn, but the libraries makes it easy to create a variety of sophisticated statistical charts. It also helps to provide color palettes for customisation of series encoding for charts. You may need to install it:

```
conda install seaborn
```

In [156...

```
import matplotlib.pyplot as plt
# Remember to use inline to get your plots in the notebook
%matplotlib inline
```

In [157...

```
# special matplotlib command for global plot configuration
from matplotlib import rcParams
import matplotlib.cm as cm
import matplotlib as mpl
import matplotlib.style

from matplotlib.colors import ListedColormap

dark2_colors = ['#1b9e77', '#d95f02', '#7570b3', '#e7298a', '#66a61e', '#e6ab02', '#a6761d', '#666666']
dark2_cmap = ListedColormap(dark2_colors)
```

```
def set_mpl_params():
    rcParams['figure.figsize'] = (10, 6)
    rcParams['figure.dpi'] = 100
    rcParams['axes.prop_cycle'].by_key()['color'][1]
    rcParams['lines.linewidth'] = 2
    rcParams['axes.facecolor'] = 'white'
    rcParams['font.size'] = 12
    rcParams['patch.edgecolor'] = 'white'
    rcParams['patch.facecolor'] = dark2_colors[0]
    rcParams['font.family'] = 'StixGeneral'
```

```
set_mpl_params()
mpl.style.use('seaborn-v0_8-dark-palette')
```

```
In [158... import seaborn as sns #If you don't have seaborn, use - conda install seaborn
current_palette = sns.color_palette()
sns.palplot(current_palette)
```

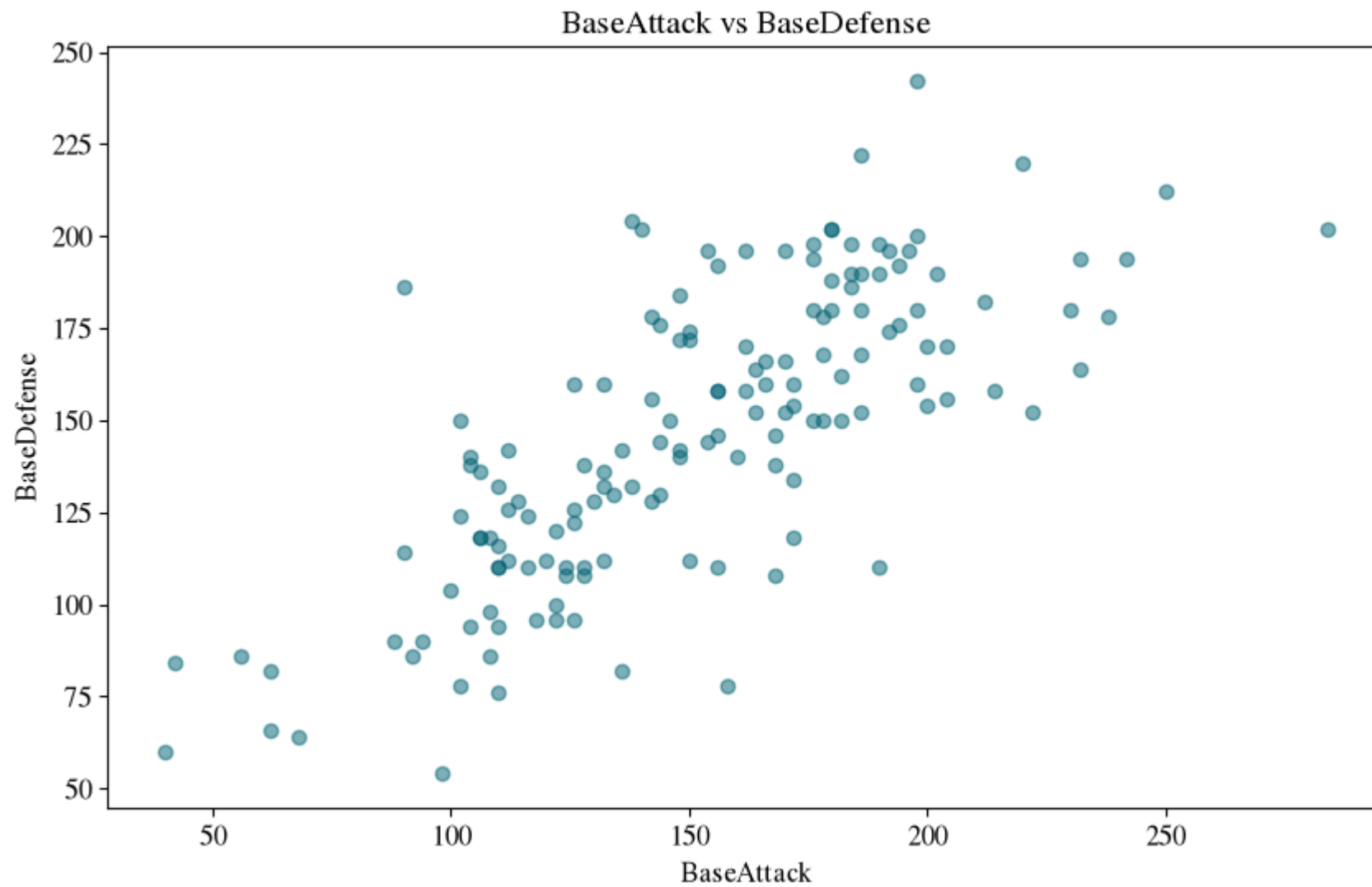


```
In [159... print (current_palette)
```

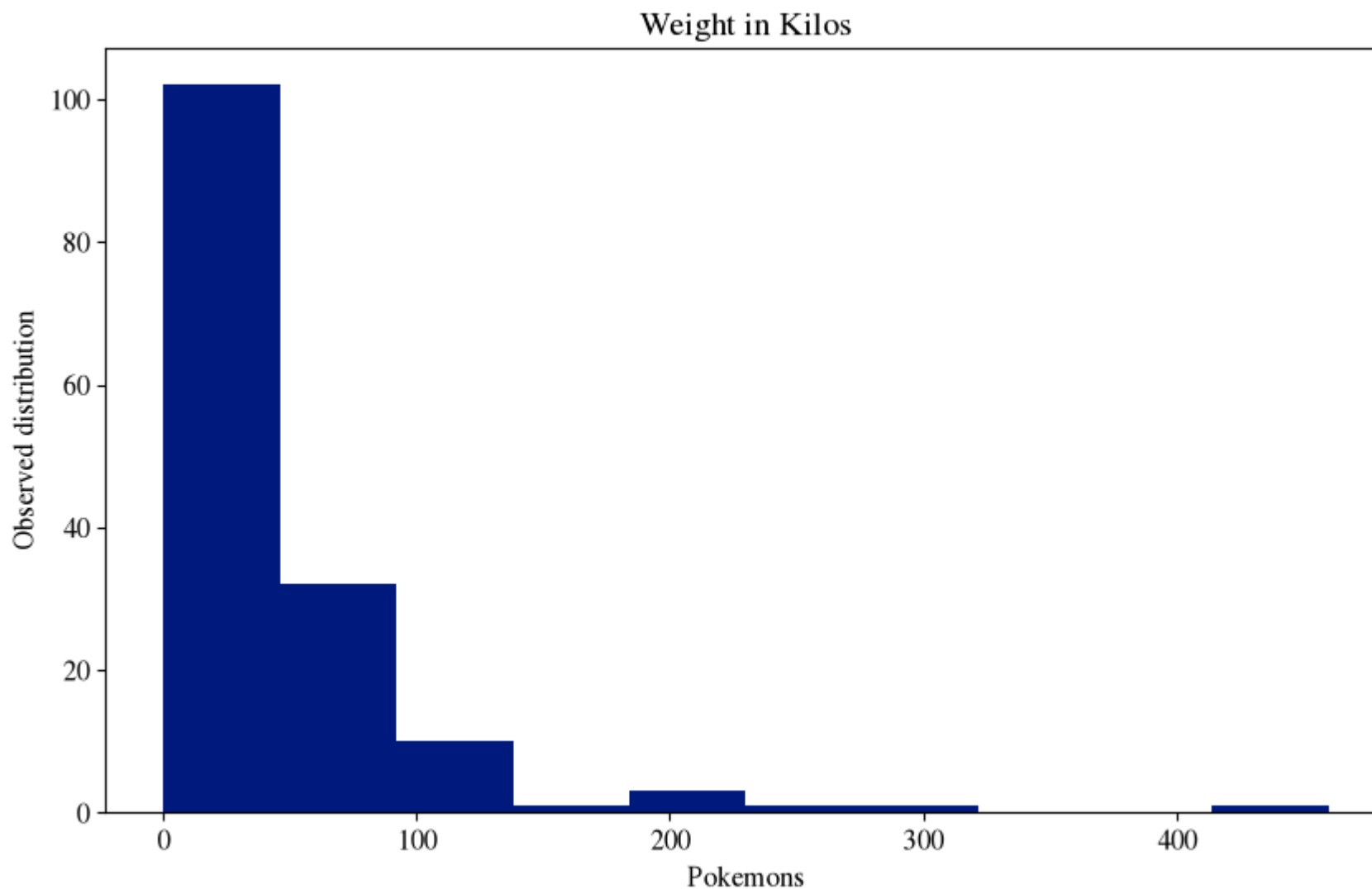
```
[(0.0, 0.10980392156862745, 0.4980392156862745), (0.00392156862745098, 0.4588235294117647, 0.09019607843137255), (0.5490196078431373, 0.03529411764705882, 0.0), (0.4627450980392157, 0.0, 0.6313725490196078), (0.7215686274509804, 0.5254901960784314, 0.043137254901960784), (0.0, 0.38823529411764707, 0.4549019607843137)]
```

[matplotlib Scatter plot](#)

```
In [160... fig=plt.figure()
plt.scatter(pokemonData.BaseAttack, pokemonData.BaseDefense, c=[current_palette[5]], alpha=0.5)
axis = fig.gca() #get current axis
axis.set_title('BaseAttack vs BaseDefense')
axis.set_xlabel('BaseAttack')
axis.set_ylabel('BaseDefense')
plt.show()
```



```
In [161... plt.hist(pokemonData.PokedexWeightKg)
plt.title('Weight in Kilos')
plt.xlabel('Pokemons')
plt.ylabel('Observed distribution');
#plt.show()
```



Multiplots

For presenting multiple series to compare distributions, trends and correlations, we can use multiple plots

```
In [162]: fig, axes=plt.subplots(figsize=(10,6), nrows=2, ncols=2)
#axes[0][0].plot(pokemonData.BaseStamina, pokemonData.PokedexWeightKg)
axes[0][0].scatter(pokemonData.WeightStdDev, pokemonData.HeightStdDev)
axes[0][0].set_xlabel("WeightStdDev")
axes[0][0].set_ylabel("HeightStdDev")

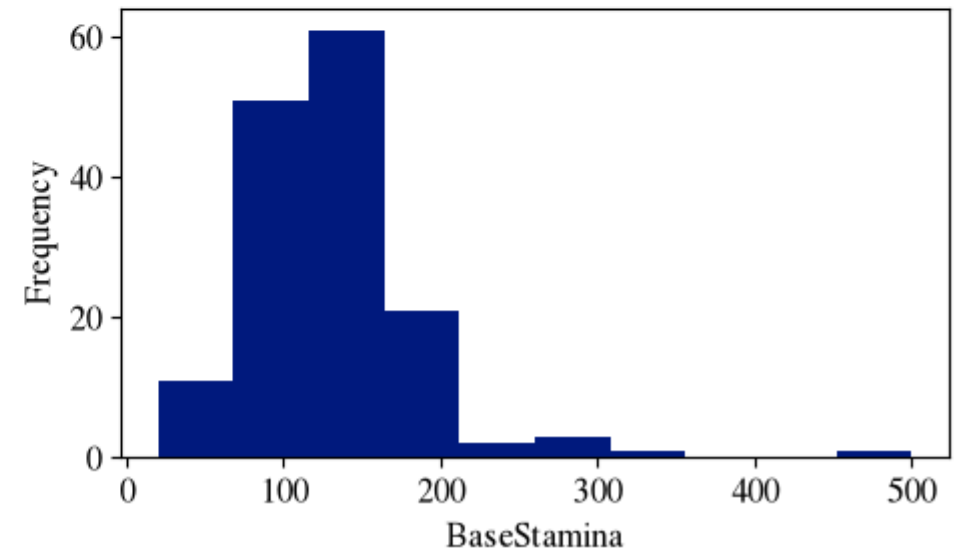
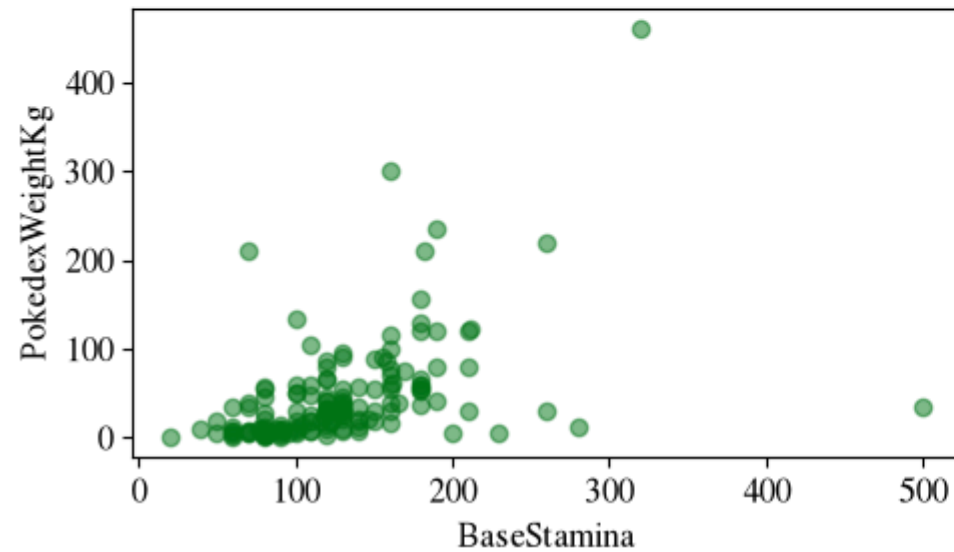
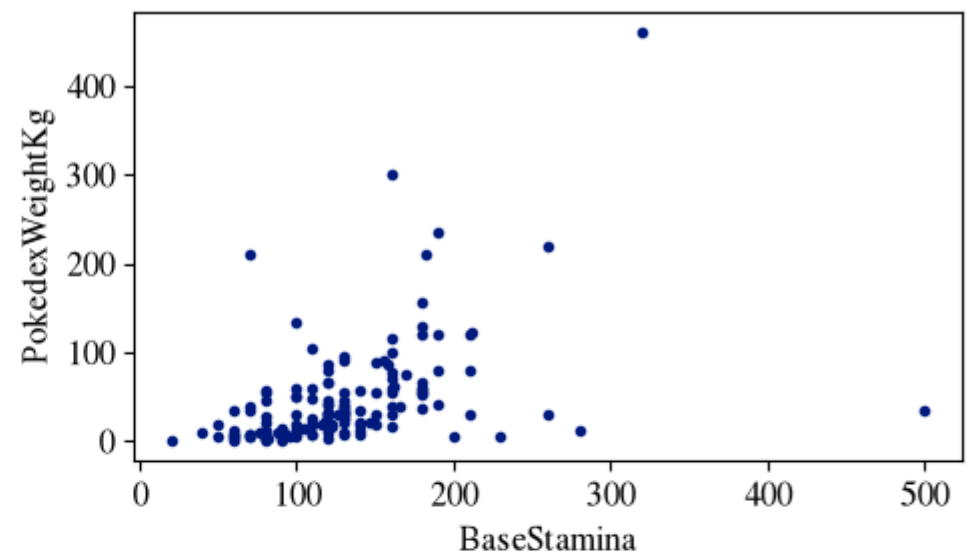
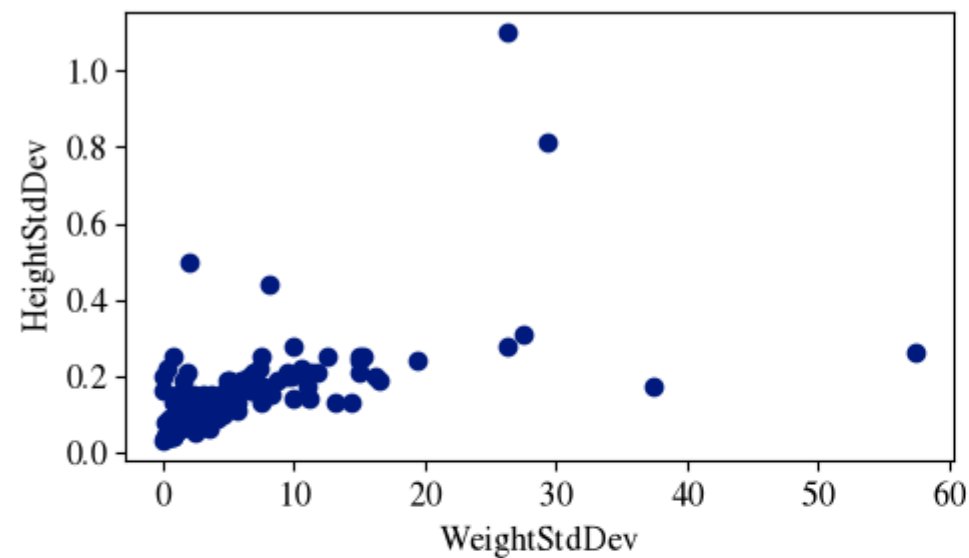
axes[0][1].plot(pokemonData.BaseStamina, pokemonData.PokedexWeightKg, '.')
axes[0][1].set_xlabel("BaseStamina")
```

```
axes[0][1].set_ylabel("PokedexWeightKg")

axes[1][0].scatter(pokemonData.BaseStamina, pokemonData.PokedexWeightKg, c=current_palette[1], alpha=0.5)
axes[1][0].set_xlabel("BaseStamina")
axes[1][0].set_ylabel("PokedexWeightKg")

axes[1][1].hist(pokemonData.BaseStamina)
axes[1][1].set_xlabel("BaseStamina")
axes[1][1].set_ylabel("Frequency")

fig.tight_layout()
plt.show()
```

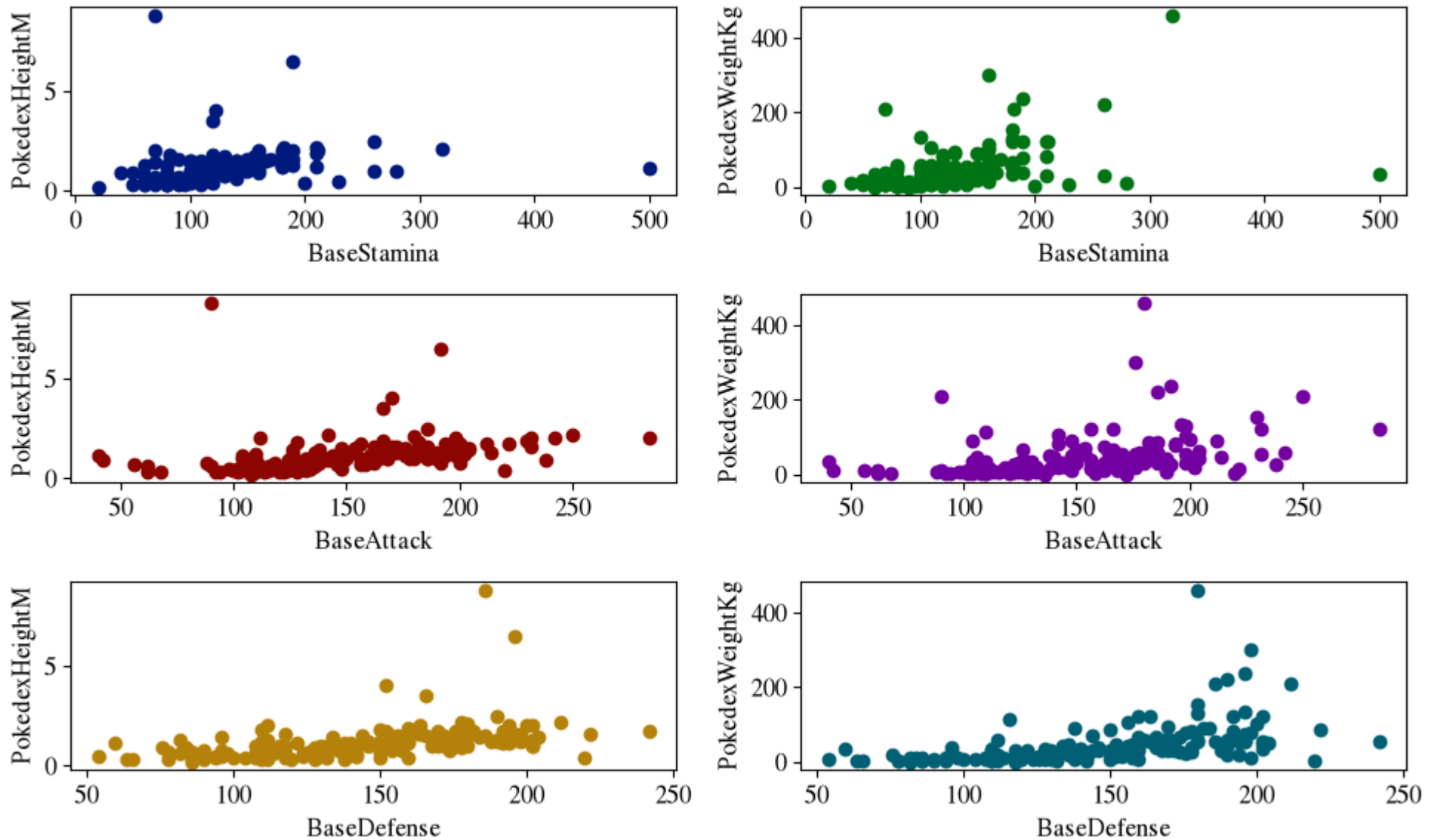


In [163]...

```
xtraits=['BaseStamina','BaseAttack','BaseDefense']
ytraits=['PokedexHeightM','PokedexWeightKg']

fig, axes=plt.subplots(figsize=(10,6), nrows=len(xtraits), ncols=len(ytraits))
k=0
for i, xtrait in enumerate(xtraits):
    for j, ytrait in enumerate(ytraits):
        axes[i][j].scatter(pokemonData[xtrait],pokemonData[ytrait], c=[current_palette[k]])
        axes[i][j].set_xlabel(xtrait)
        axes[i][j].set_ylabel(ytrait)
        k = k+1
```

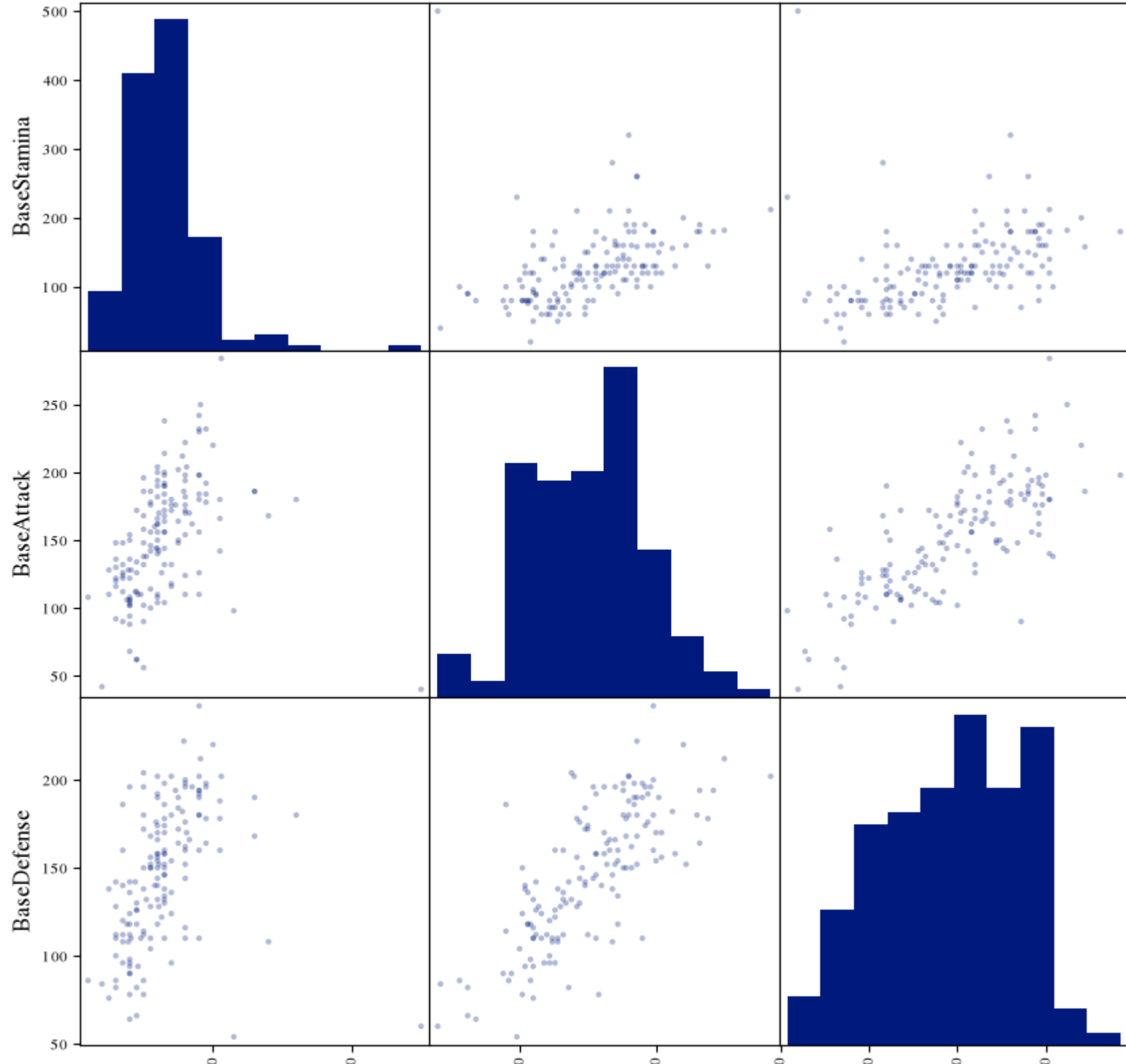
```
fig.tight_layout()
plt.show()
```



Pandas plotting

Pandas also can be used for easy plotting of multiple series. You can find more information [here](#)

```
In [164... from pandas.plotting import scatter_matrix
scatter_matrix(pokemonData[xtraits], alpha=0.3, figsize=(10, 10));
```



BaseStamina BaseAttack BaseDefense

Boxplots

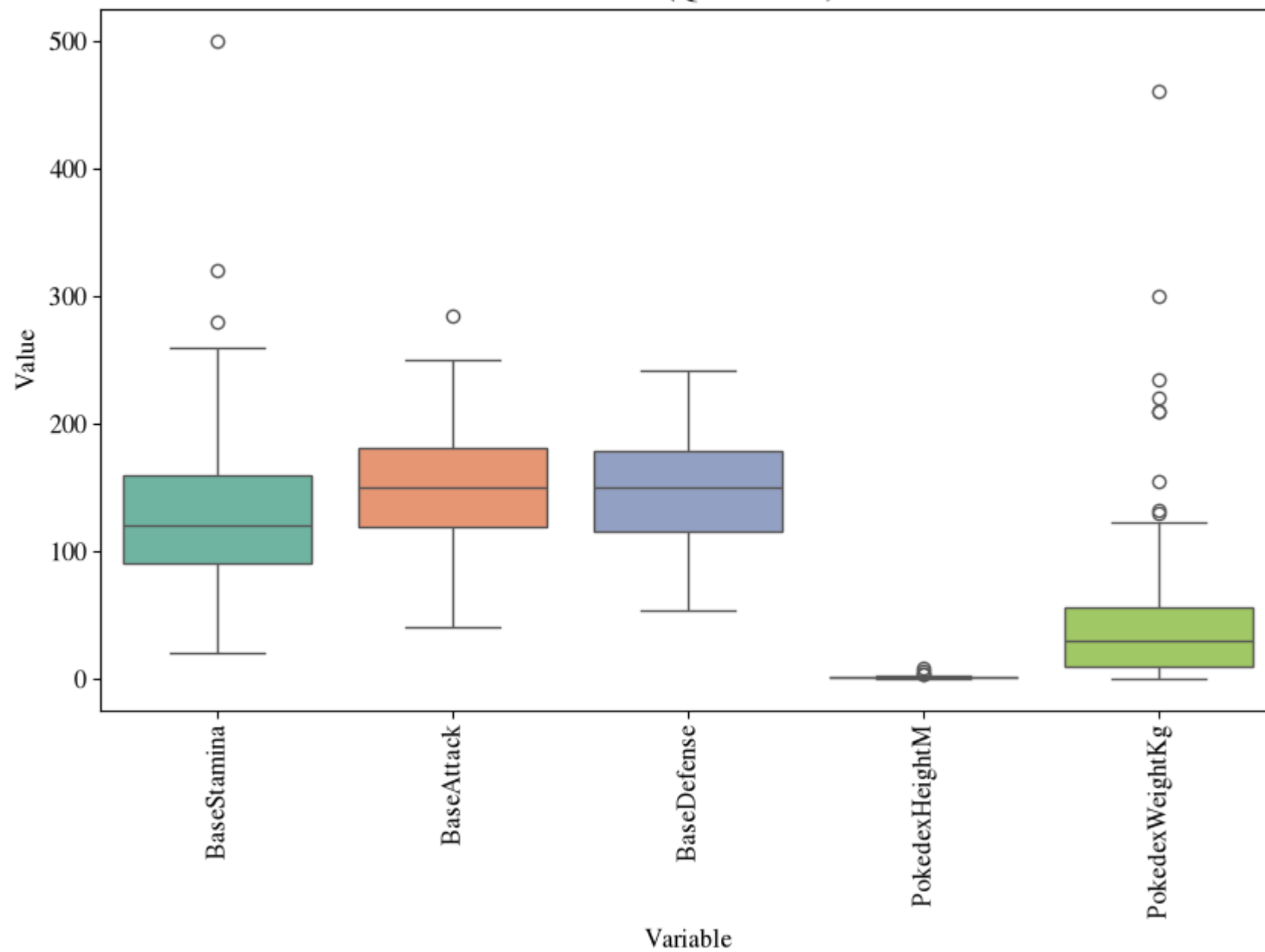
Boxplots can be used to show difference between distributions of variables

```
In [165... xtraits=['BaseStamina', 'BaseAttack', 'BaseDefense']
ytraits=['PokedexHeightM', 'PokedexWeightKg']

[xtraits.append(i) for i in ytraits]
plt.subplots(figsize=(10,6))
ax = sns.boxplot(data=pokemonData[xtraits], palette="Set2")
plt.title('Variables (Quantitative)')
plt.ylabel('Value')
plt.xlabel('Variable')
plt.xticks(rotation=90)

# for more about the warning message, you can click on the blue underlined message and trace it back.
# ultimately, you will land here: https://github.com/pandas-dev/pandas/blob/v2.1.0/pandas/core/dtypes/common.py#L469-L511
_=plt.show
```

Variables (Quantitative)



In [166... `pokemonData[xtraits].dtypes`

```
Out[166... BaseStamina      int64
BaseAttack      int64
BaseDefense      int64
PokedexHeightM  float64
PokedexWeightKg  float64
dtype: object
```

Parallel coordinates

And parallel coordinates plots can indicate how characteristics vary across groups

```
In [167... pokemonData[xtraits].describe()
```

```
Out[167... 
```

	BaseStamina	BaseAttack	BaseDefense	PokedexHeightM	PokedexWeightKg
count	151.000000	151.000000	151.000000	151.000000	151.000000
mean	128.423841	151.152318	146.370861	1.194702	45.951656
std	57.180234	42.994844	39.253300	0.962621	59.447988
min	20.000000	40.000000	54.000000	0.200000	0.100000
25%	90.000000	119.000000	115.000000	0.700000	9.900000
50%	120.000000	150.000000	150.000000	1.000000	30.000000
75%	160.000000	181.000000	178.000000	1.500000	56.250000
max	500.000000	284.000000	242.000000	8.800000	460.000000

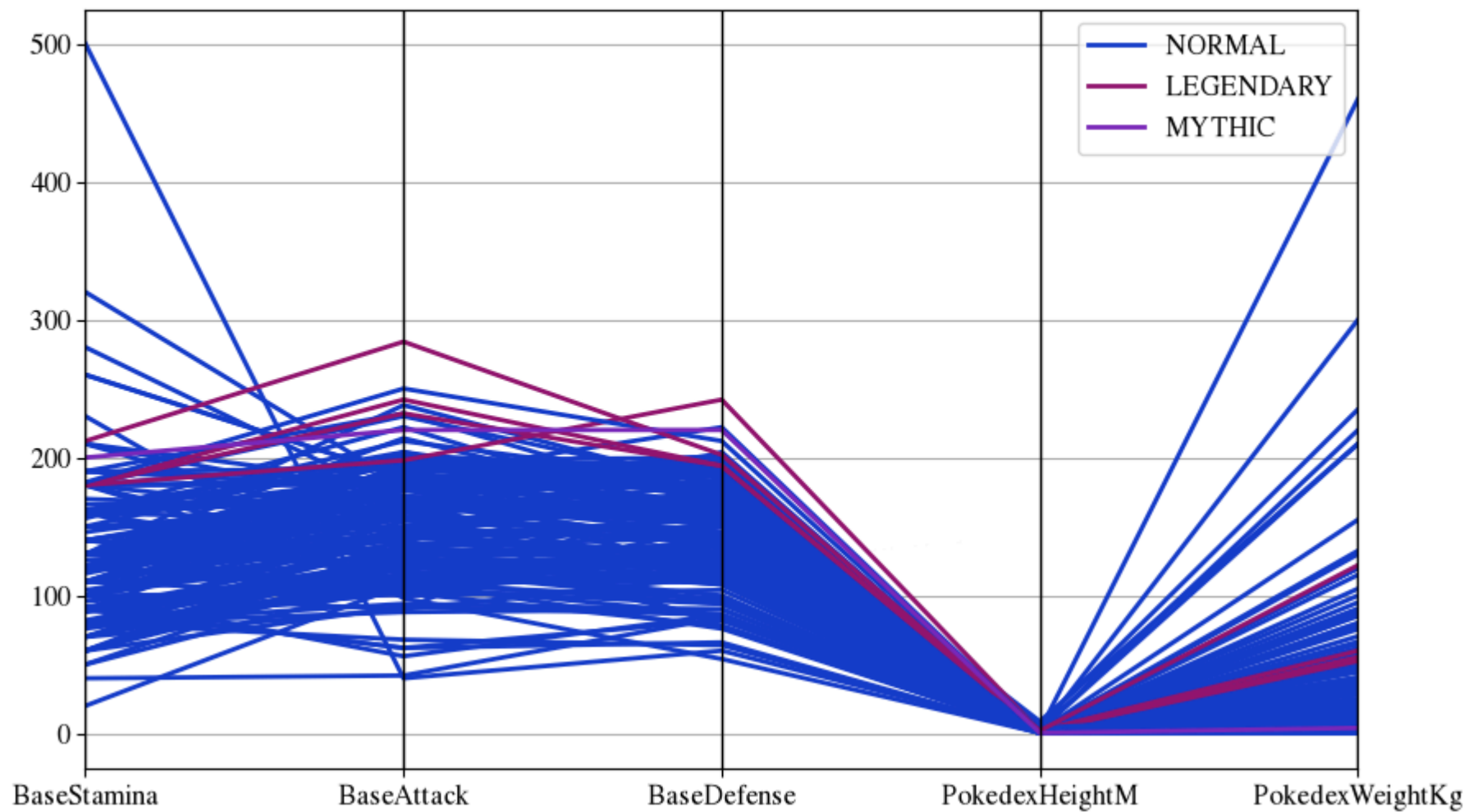
```
In [168... pokemonData['EvolutionPips'].value_counts()
```

```
Out[168... EvolutionPips
NORMAL      146
LEGENDARY    4
MYTHIC       1
Name: count, dtype: int64
```

```
In [169... from pandas.plotting import parallel_coordinates

xtraits.append('EvolutionPips')
plt.figure()
_=parallel_coordinates(pokemonData[xtraits], 'EvolutionPips')

#See https://datavizcatalogue.com/methods/parallel\_coordinates.html for good description
```



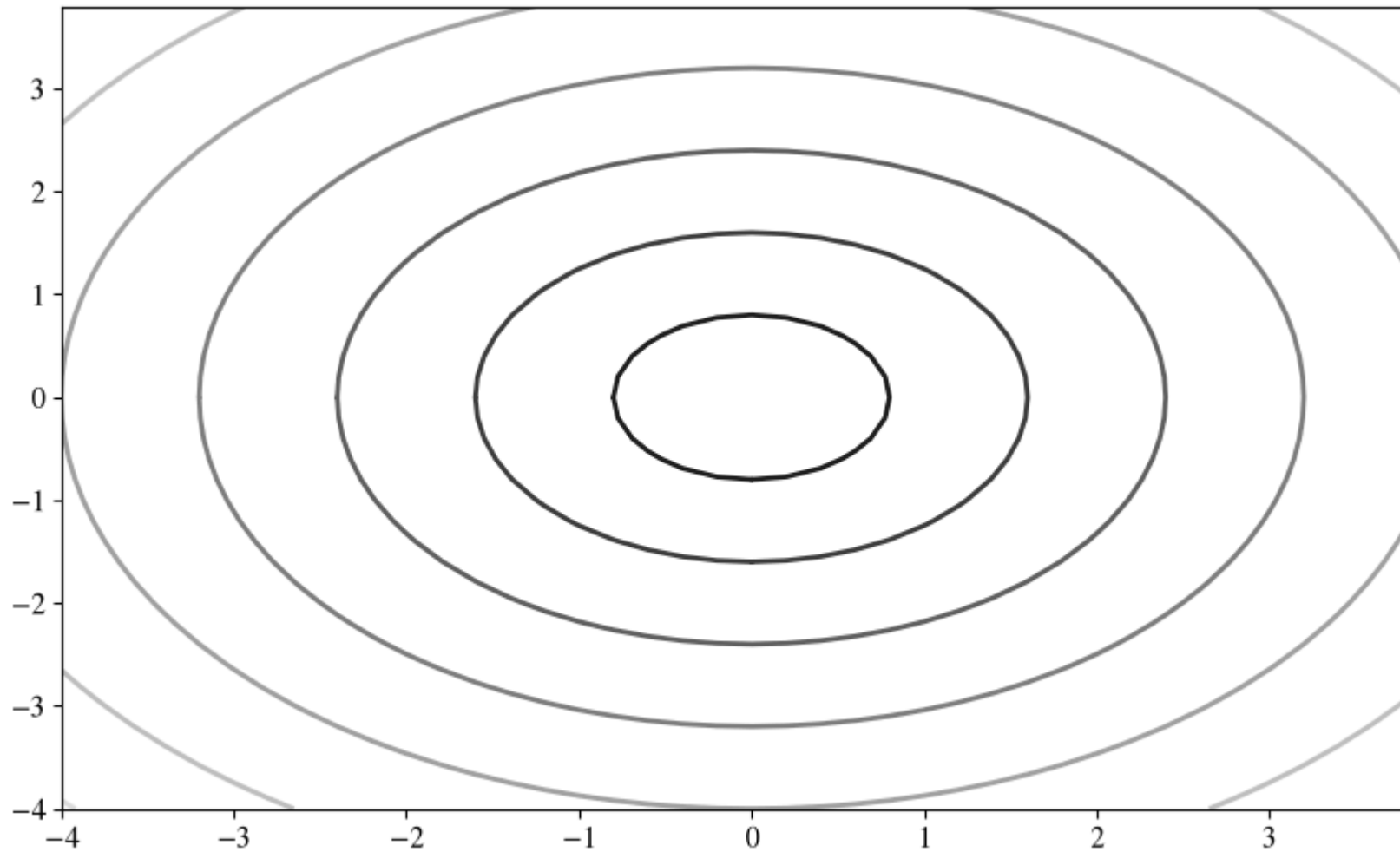
Other interesting plots

Matplotlib does 3D, surface density plots and contour plots that we will find useful for presenting our results.

Contour plots

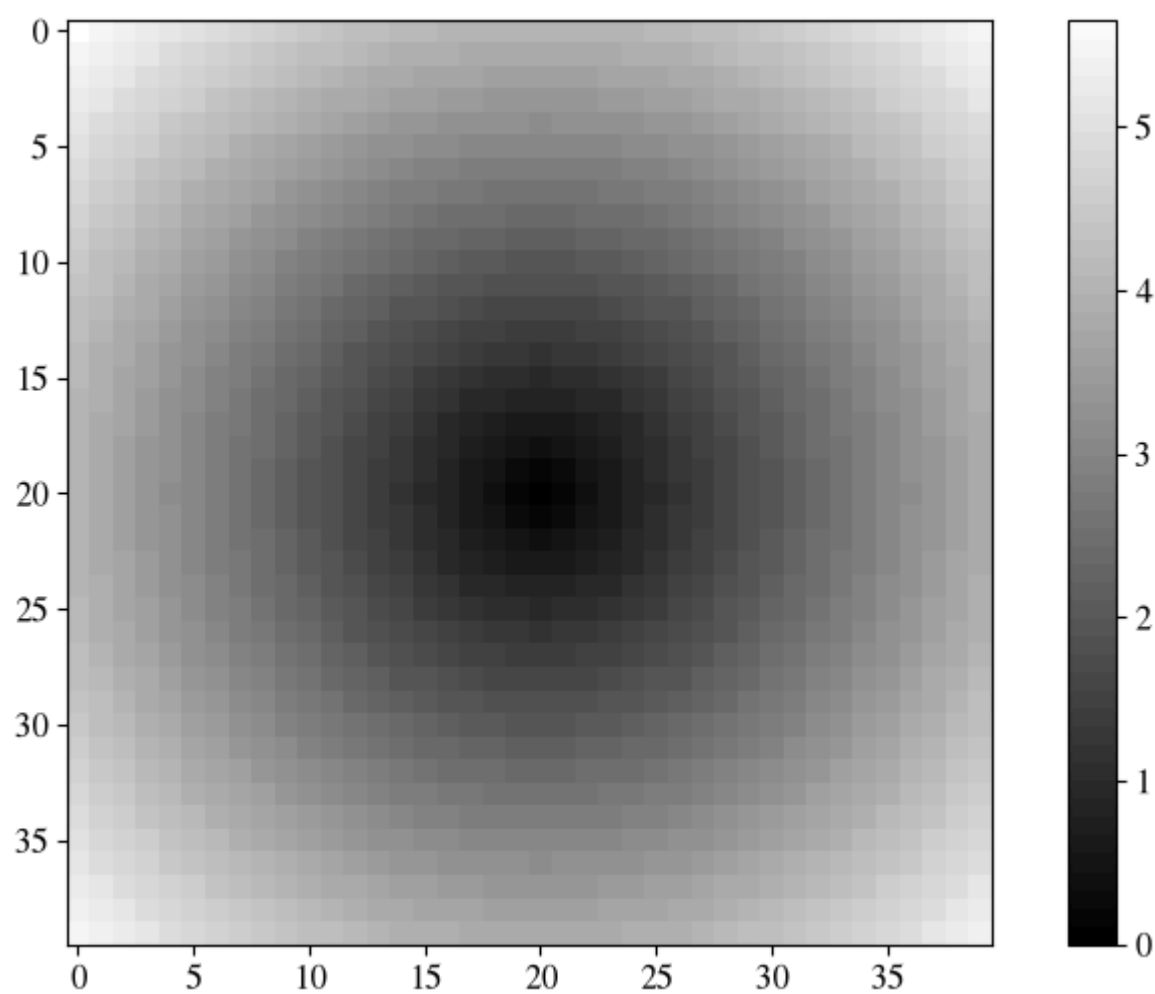
```
In [170... # Contour plots
x = np.arange(-4, 4, 0.2);
y = np.arange(-4, 4, 0.2);
xs, ys = np.meshgrid(x,y)
z = np.sqrt(xs**2 + ys**2)
```

```
plt.contour(xs, ys, z)  
plt.show() #See Khanacademy video on Contour plot (by Grant Sanderson)
```



Surface density plots

```
In [171... plt.imshow(z, cmap=plt.cm.gray); plt.colorbar()  
plt.show()
```



3-D Surface plots

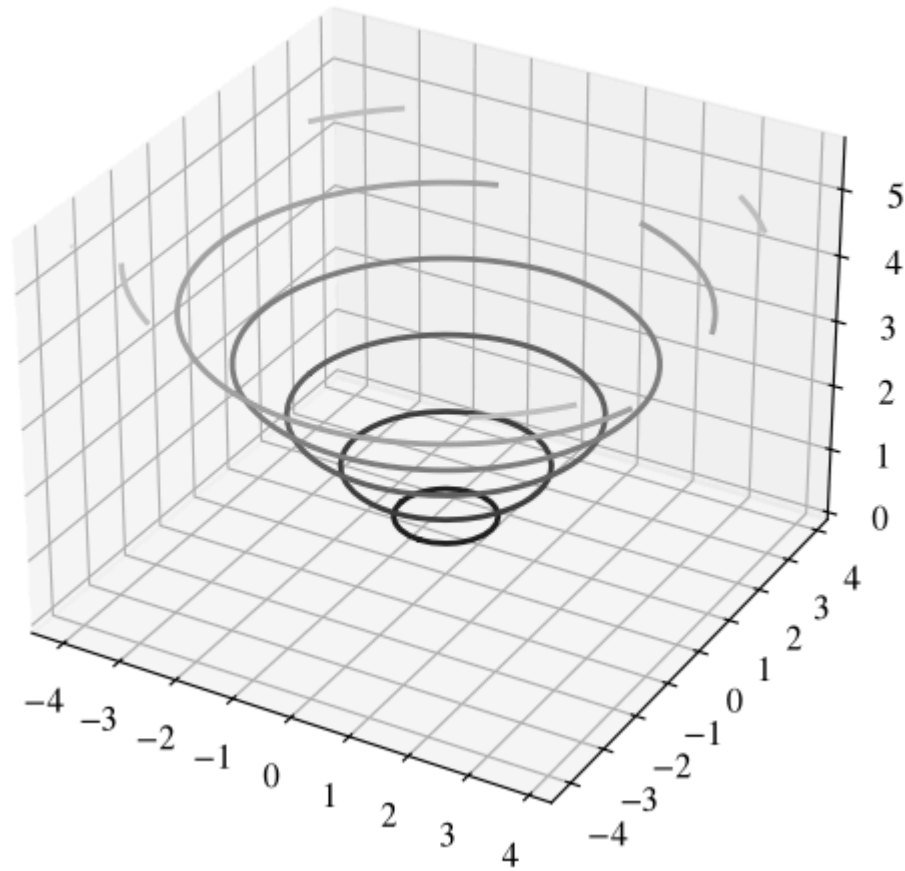
Some interesting documentation to get this right is [here for wireframes and surface plots](#) and [here for marking up](#)

```
In [172... # 3-D Surface plot
from mpl_toolkits.mplot3d import axes3d
import matplotlib.pyplot as plt

fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')

#ax.plot_wireframe(xs,ys,z, rstride=10, cstride=10)
#ax.plot_surface(xs,ys,z, rstride=10, cstride=10)
ax.contour(xs, ys, z)
```

```
plt.show()
```



[back to top](#)